

Fully Dynamic Spectral and Cut Sparsifiers for Directed Graphs

Yibin Zhao 

University of Toronto, Canada

Abstract

Recent years have seen extensive research on directed graph sparsification. In this work, we initiate the study of fast fully dynamic spectral and cut sparsification algorithms for directed graphs.

We introduce a new notion of spectral sparsification called degree-balance preserving spectral approximation, which maintains the difference between the in-degree and out-degree of each vertex. The approximation error is measured with respect to the corresponding undirected Laplacian. This notion is equivalent to direct Eulerian spectral approximation when the input graph is Eulerian. Our algorithm achieves an amortized update time of $O(\varepsilon^{-2} \cdot \text{polylog}(n))$ and produces a sparsifier of size $O(\varepsilon^{-2}n \cdot \text{polylog}(n))$. Additionally, we present an algorithm that maintains a constant-factor approximation sparsifier of size $O(n \cdot \text{polylog}(n))$ against an adaptive adversary for $O(\text{polylog}(n))$ -partially symmetrized graphs, a notion introduced in [Kyng-Meierhans-Probst Gutenberg '22]. A β -partial symmetrization of a directed graph $\vec{G}$ is the union of $\vec{G}$ and $\beta \cdot G$, where G is the corresponding undirected graph of $\vec{G}$. This algorithm also achieves a polylogarithmic amortized update time.

Moreover, we develop a fully dynamic algorithm for maintaining a cut sparsifier for β -balanced directed graphs, where the ratio between weighted incoming and outgoing edges of any cut is at most β . This algorithm explicitly maintains a cut sparsifier of size $O(\varepsilon^{-2}\beta n \cdot \text{polylog}(n))$ in worst-case update time $O(\varepsilon^{-2}\beta \cdot \text{polylog}(n))$.

2012 ACM Subject Classification Theory of computation $\rightarrow$ Dynamic graph algorithms; Theory of computation $\rightarrow$ Sparsification and spanners

Keywords and phrases Dynamic graph algorithm, Spectral graph theory, Sparsifier

Digital Object Identifier 10.4230/LIPIcs.ICALP.2026.157

Category Track A: Algorithms, Complexity and Games

Related Version *Full Version*: <https://arxiv.org/abs/2507.19632> [48]

Acknowledgements The author would like to thank Sushant Sachdeva and Richard Peng for helpful discussions.

1 Introduction

Designing spectral algorithms for directed graphs is a major frontier in algorithmic spectral graph theory. These algorithms have found numerous applications, ranging from fast algorithms for processing Markov chains [21, 3] to deterministic low-space computation [4]. Spectral sparsification of directed (Eulerian) graphs has emerged as a central topic in this area, with considerable recent advances [20, 16, 5, 44, 32, 38]. In particular, faster spectral sparsification algorithms that produce sparser directed Eulerian sparsifiers directly lead to faster computation of approximate Personalized PageRank vectors [21], stationary distributions, hitting times, escape probabilities of random walks [21], and approximate Perron vectors [3] through efficient directed Eulerian Laplacian solvers [20, 42].

In this work, we initiate the study of spectral and cut sparsification for dynamically changing directed graphs. We believe that our dynamic algorithms will serve as the foundation for efficient dynamic computations of the aforementioned problems.



© Yibin Zhao;

licensed under Creative Commons License CC-BY 4.0

53rd International Colloquium on Automata, Languages, and Programming (ICALP 2026).

Editors: Sayan Bhattacharya, Danupon Nanongkai, Michael Benedikt, and Gabriele Puppis;

Article No. 157; pp. 157:1–157:24



Leibniz International Proceedings in Informatics

Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany



Unlike undirected spectral sparsification, developing a useful notion of directed spectral or cut sparsification has been particularly challenging. Consider the complete directed bipartite graph where every node on one side of the bipartition has a directed edge to every node on the other side. Any sparsifier that approximately preserves all directed cuts in such a graph cannot delete any edges. For cut sparsification, Cen, Cheng, Panigrahi, and Sun [13] restricted the sparsification guarantees to only apply to cuts with values close to their corresponding cuts in the reverse direction.

► **Definition 1** (β -balanced directed cut approximation). $\vec{H} = (V, E_{\vec{H}}, \mathbf{w}_{\vec{H}})$ is a (β, ε) -balanced directed cut approximation of $\vec{G} = (V, E, \mathbf{w})$ if for all non-trivial cuts $(U, V \setminus U)$ satisfying $\frac{1}{\beta} \mathbf{w}(V \setminus U, U) \leq \mathbf{w}(U, V \setminus U) \leq \beta \mathbf{w}(V \setminus U, U)$, $\vec{H}$ satisfies that

$$(1 - \varepsilon) \mathbf{w}(U, V \setminus U) \leq \mathbf{w}_{\vec{H}}(U, V \setminus U) \leq (1 + \varepsilon) \mathbf{w}(U, V \setminus U).$$

This notion of β -balanced cut sparsification is motivated by applications in solving flows and cuts problems [24, 34], as the residual graphs that appear in the intermediate steps of the maxflow algorithms are naturally balanced. These sparsifiers have found further application in recent work by Goranci, Henzinger, Räcke, and Sricharan [29], who used them to achieve the first dynamic incremental maxflow algorithm with polylogarithmic amortized update time for dense graphs ($m = \Omega(n^2)$).

Cohen, Kelner, Peebles, Peng, Rao, Sidford, and Vladu [20] addressed the directed spectral sparsification problem by focusing on directed Eulerian graphs (where every vertex has equal weighted in-degree and out-degree). This requirement might seem overly restrictive, since all directed cuts of an Eulerian graph must have a balance factor $\beta = 1$. However, Cohen et al. demonstrated that their sparsification approach suffices for developing fast solvers for all directed Laplacian linear systems (even those not corresponding to an Eulerian graph). In this paper, we propose a relaxed notion of directed spectral approximation closely related to that of [20], where the sparsifier only needs to maintain the degree balances.

► **Definition 2** (Degree balance preserving directed spectral approximation). $\vec{H} = (V, E_{\vec{H}}, \mathbf{w}_{\vec{H}})$ is a ε -degree balance preserving directed spectral approximation of $\vec{G} = (V, E, \mathbf{w})$ if for the corresponding undirected $G \stackrel{\text{def}}{=} \text{und}(\vec{G})$,

$$\forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^V, \quad \left| \mathbf{x}^\top (\vec{\mathbf{L}}_{\vec{G}} - \vec{\mathbf{L}}_{\vec{H}}) \mathbf{y} \right| \leq \varepsilon \sqrt{\mathbf{x}^\top \mathbf{L}_G \mathbf{x} \cdot \mathbf{y}^\top \mathbf{L}_G \mathbf{y}}. \quad (1)$$

This definition of directed spectral approximation has been implicitly used in almost all previous works in the directed Laplacian literature (e.g. [20, 19, 16, 37, 44, 32]). We note that the degree balance preserving requirement is implicitly ensured by (1): if the degree balance is not preserved, then there are some vectors $\mathbf{x}, \mathbf{y}$ such that the left hand side of (1) is non-zero while the right hand side is zero (see Lemma 10). The relaxation to any directed graph is necessary for dynamically changing graphs, as modifying from one directed Eulerian graph to another requires at least 3 individual edge updates. Whenever a dynamic directed graph $\vec{G}$ becomes Eulerian, our degree balance preservation requirement naturally ensures that the dynamically maintained sparsifier is also Eulerian.

For both directed balanced cut sparsification and degree balance preserving spectral sparsification, we ask the natural question of whether a fully dynamic algorithm with polylogarithmic update time exists for maintaining a nearly-linear size sparsifier (i.e., the number of edges in the sparsifier is at most $O(n \cdot \text{poly}(\varepsilon^{-1}, \log n))$ for directed graphs. Previous work by Abraham, Durfee, Koutis, Krinninger, and Peng [1] established the existence of such dynamic algorithms for undirected graphs. In this work, we answer this question affirmatively for the directed case.

1.1 Our results

Our first result is a fully dynamic algorithm for degree balance preserving directed spectral sparsification with polylogarithmic amortized update time.

► **Theorem 3** (Explicit dynamic directed spectral sparsifier). *Given a directed graph $\vec{G}$ on vertices V and with polynomially bounded edge weights, we can maintain explicitly a graph $\vec{H}$ on vertices $V \cup X$ such that the Schur complement $\text{Sc}(\vec{H}, V)$ onto the original vertices is a ε -degree balance preserving directed sparsifier. $\vec{H}$ has size $\tilde{O}(\varepsilon^{-2}n)$ ¹ and satisfies $|X| = \tilde{O}(n)$. The algorithm has amortized recourse and update time $\tilde{O}(\varepsilon^{-2})$ per edge insertion or deletion and works against an oblivious adversary.*

Our algorithm explicitly maintains a sparsifier at the expense of $\tilde{O}(n)$ extra vertices. We demonstrate in Lemma 13 (Section 3) that these extra vertices do not affect the usefulness of our theorem. Crucially, having an explicit sparsifier with low recourse enables further dynamic algorithms using our algorithm as a dynamic subroutine. A version of this theorem that exactly preserves both in-degrees and out-degrees is presented in Theorem 24. We also present two different algorithms that maintain *implicit* sparsifiers on the original set of vertices in the full version.

As observed by [11] for dynamic undirected spectral sparsifiers, dynamic algorithms that maintain implicit sparsifiers already support important applications. For example, one can maintain a dynamic row and column diagonally dominant (RCDD) Z-matrix² solver through a reduction to directed Eulerian spectral sparsifier by [21] and by running a non-iterative static directed Eulerian Laplacian solver (e.g., [42]) on top of the sparsifier.

► **Corollary 4** (Dynamic RCDD Z-matrix solver). *Given a dynamic RCDD Z-matrix $\mathbf{M}$ of dimension n and $\varepsilon \in (0, 1)$, we can maintain a data structure in $\tilde{O}(\varepsilon^{-2}n)$ amortized update time which supports solving the linear system $\mathbf{M}\mathbf{x} = \mathbf{b}$ for any $\mathbf{b} \in \mathbb{R}^n$ to ε -approximation in time $\tilde{O}(\varepsilon^{-2}n)$.*

Our next result, parallel to spectral sparsification, is a simple fully dynamic balanced directed cut sparsification algorithm with polylogarithmic worst-case update time.

► **Theorem 5** (Dynamic β -balanced cut sparsifier). *Given a directed graph with polynomially bounded edge weights, we can maintain explicitly a (β, ε) -balanced directed cut sparsifier of size $\tilde{O}(\varepsilon^{-2}\beta n)$ with worst-case update time $\tilde{O}(\varepsilon^{-2}\beta)$ per edge insertion or deletion against an oblivious adversary.*

Unlike spectral sparsification, cut sparsification does not require degree fixing. As a result, our data structure can maintain a cut sparsifier explicitly without extra vertices. Additionally, we present an algorithm with improved amortized recourse and update time (see Theorem 27 in Section 4). In achieving Theorem 5, we also prove that independent edge sampling based on inverse undirected edge connectivity is sufficient for balanced directed cut approximation.

► **Theorem 6** (Improved static β -balanced cut sparsifier). *There is a sparsification algorithm that, given a weighted directed graph and the edge connectivities of its corresponding undirected graph, computes a (β, ε) -balanced directed cut sparsifier with $O(\varepsilon^{-2}\beta n \log n)$ edges in linear time.*

¹ We use $\tilde{O}(\cdot)$ to hide polylogarithmic factors in n .

² A RCDD matrix is a real M-matrix with non-negative row and column sums.

Finally, we present a fully dynamic directed spectral sparsification algorithm that works against an adaptive adversary. This result requires more detailed explanation, which we provide below.

We first remind readers that our degree balance preserving sparsifier requires the approximation factor $\varepsilon < 1$. This requirement is necessary and warranted – for any directed Eulerian graph, an empty graph on the same set of vertices can serve as a constant good approximation. To the best of our knowledge, there exists no notion of directed spectral approximation that extends beyond constant approximation. This limitation is particularly problematic since the state-of-the-art adaptive undirected cut and spectral sparsifier from [11] require the multiplicative approximation factor to be at least $O(\log n)$.

Kyng, Meierhans, and Probst Gutenberg [37] circumvented this issue by considering the sparsification problem on a related graph called a *partial symmetrization*. Specifically, a β -partial symmetrization of a directed graph $\vec{G}$ is defined by $\vec{G}^{(\beta)} \stackrel{\text{def}}{=} \beta \cdot G \cup \vec{G}$, which combines the directed graph with a copy of its corresponding undirected graph $G \stackrel{\text{def}}{=} \text{und}(\vec{G})$ scaled by a factor of β . They showed that a partially symmetrized Eulerian graph serves as a good *approximate pseudoinverse* of the original directed graph (see Lemma 37).

► **Definition 7** (Approximate pseudoinverse, [21]). *For $\varepsilon \geq 0$, a square matrix $\mathbf{Z}$ is an ε -approximate pseudoinverse of square matrix $\mathbf{M}$ with respect to a Hermitian PSD matrix $\mathbf{U}$ if $\ker(\mathbf{U}) \subseteq \ker(\mathbf{M}) = \ker(\mathbf{M}^\top) = \ker(\mathbf{Z}) = \ker(\mathbf{Z}^\top)$, and*

$$\|\mathbf{P}_{\text{im}(\mathbf{M})} - \mathbf{Z}\mathbf{M}\|_{\mathbf{U} \rightarrow \mathbf{U}} \leq \varepsilon,$$

where $\mathbf{P}_{\text{im}(\mathbf{M})}$ is the identity matrix on the image of $\mathbf{M}$.

Moreover, when β is large, the directed portion of $\vec{G}^{(\beta)}$ can be sparsified much more aggressively under the notion of degree balance preserving spectral approximation, since

$$\left\| \mathbf{L}_{\vec{G}^{(\beta)}}^{\frac{1}{2}} (\vec{\mathbf{L}}_{\vec{G}} - \vec{\mathbf{L}}_{\vec{H}}) \mathbf{L}_{\vec{G}^{(\beta)}}^{\frac{1}{2}} \right\|_{\text{op}} = \frac{1}{2\beta + 1} \left\| \mathbf{L}_{\vec{G}}^{\frac{1}{2}} (\vec{\mathbf{L}}_{\vec{G}} - \vec{\mathbf{L}}_{\vec{H}}) \mathbf{L}_{\vec{G}}^{\frac{1}{2}} \right\|_{\text{op}}.$$

Further sparsification can also be performed on the undirected portion of the graph $\vec{G}^{(\beta)}$ to ultimately produce a sparse approximation. These layers of approximation allow them to build a sparse preconditioner chain, which can be applied to solve directed Eulerian Laplacian systems using iterative solvers. See Section 5 for more details. Our dynamic algorithm efficiently maintains these sparsification layers under adaptive edge insertions and deletions.

► **Theorem 8** (Adaptive directed spectral sparsification quadruple). *Given a directed graph $\vec{G}$ on vertices V and with polynomially bounded edge weights, we can maintain explicitly a set of 3 directed graphs $\vec{G}_1 = \vec{G}^{(\beta)}$, $\vec{G}_2$, $\vec{G}_3$ with $\vec{G}_0 = \vec{G}$ and $V(\vec{G}_2) = V \cup X$ and $V(\vec{G}_3) = V \cup X \cup Y$. For $\vec{G}_2 \stackrel{\text{def}}{=} \text{Sc}(\vec{G}_2, V)$ and $\vec{G}_3 \stackrel{\text{def}}{=} \text{Sc}(\vec{G}_3, V)$, they satisfy that*

1. *when $\vec{G}$ is Eulerian $\vec{G}_i$ is a $1 - \frac{1}{\text{polylog}(n)}$ -approximate pseudoinverse of $\vec{G}_{i-1}$ for all $i = 1, 2, 3$ and degree balance preserving with respect to $\vec{G}$,*
2. *$\vec{G}_3$ has size $\tilde{O}(n)$ and $\vec{G}_2$ has size $O(m)$.*

The algorithm works against an adaptive adversary, has preprocessing time $\tilde{O}(m)$ and amortized update time $\tilde{O}(1)$.

In this paper, we provide an overview of our algorithms and techniques. We refer readers to the full version of our paper for formal statements and detailed proofs.

1.2 Related works

Static directed graph sparsification

The notion of Eulerian Laplacian spectral sparsification was introduced by [20] to construct fast directed Eulerian Laplacian linear system solvers. These solvers have applications in various computational problems related to random walks on general directed graphs, through reductions given in [21, 3]. Cohen et al. [20] gave the first $\tilde{O}(m)$ time algorithm for constructing Eulerian sparsifiers with $\tilde{O}(\varepsilon^{-2}n)$ edges, based on expander decompositions.

As an alternative approach, Chu et al. [16] introduced the short cycle decomposition technique and used it to give a polynomial-time algorithm for computing sparsifiers with significantly improved sparsity. This approach was subsequently improved by the work of [40, 41, 44]. Notably, [44] also demonstrated an improved existential result on the sparsity of Eulerian sparsifiers by leveraging techniques from matrix discrepancy theory [7]. This discrepancy approach was earlier introduced by Jambulapati, Reis, and Tian [31] for constructing undirected degree preserving spectral sparsifiers with optimal sparsity in almost linear time.

Recently, Jambulapati et al. [32] achieved state-of-the-art Eulerian sparsification results for both efficient sparsification and existential sparsity using a combination of techniques: effective resistance decomposition and sampling by electrical circulations. When combined with the reduction by Peng and Song [42], they provided the state-of-the-art Eulerian Laplacian solver that runs in $\tilde{O}(m \log^3 n + n \log^6 n)$ time³. Lau, Wang, and Zhou [38] gave a derandomized Eulerian sparsification algorithm using the “deterministic discrepancy walk” approach from [43]. They extended their sparsification result to satisfy a stronger notion of directed spectral approximation known as “singular value (SV) approximation” [5].

For directed cut sparsification, the notion of balanced cut approximation was introduced by Cen et al. [13] for sparsification of directed graphs where each cut has approximately the same weight as the cut in the reverse direction. Efficient cut sparsification algorithms have applications in minimum cut approximations for such balanced graphs [24]. Cen et al. designed an algorithm for constructing cut sparsifiers with $O(\varepsilon^{-2}\beta n \log n)$ edges for balance factor β by sampling using the inverse undirected edge strength, introduced in [9] for undirected cut sparsification. This upper bound was later proven to be nearly tight by Chen et al. [15], who gave a bit complexity lower bound of $\Omega(\varepsilon^{-2}\beta n)$. Theorem 6 complements the results of [13] by showing that sampling with probability inversely proportional to undirected edge connectivities, similar to [27] for undirected cut sparsification, achieves the same sparsity guarantee.

Recently, Goranci, Henzinger, Räcke, and Sricharan [29] independently discovered a similar generalization to balanced cut approximation using the framework of [27]. They proved a stronger result showing that any undirected edge connectivity measure is suitable for sparsification of balanced directed cuts. One key difference is that their general result has a slightly worse sparsity guarantee compared to ours that specifically uses undirected edge connectivity. Their sparsification algorithm works in conjunction with a dynamic NI index algorithm to dynamically sparsify residual networks in incremental maxflow problems.

³ We use $\tilde{O}(\cdot)$ to hide polyloglog factors in n .

Dynamic graph sparsification

Dynamic graph sparsification problems have been studied extensively for undirected graphs. There is a long line of work [6, 23, 8, 25, 10, 11, 14, 47, 18] on maintaining spanners – graph sparsifiers that approximately preserve distances – with various assumptions about updates (amortized or worst-case) and adversaries (oblivious or adaptive).

Abraham, Durfee, Koutis, Krinninger, and Peng [1] initiated the study of dynamic undirected spectral and cut sparsifications. They developed fully dynamic spectral sparsifiers with $\tilde{O}(\varepsilon^{-2}n)$ edges and $\tilde{O}(\varepsilon^{-2})$ amortized update time against oblivious adversaries by adapting the static spectral sparsification algorithm of Koutis and Xu [36]. Their algorithm maintains a bundle of dynamic spanners [8] that can certify edges with small effective resistances, the central value for undirected spectral sparsification. By swapping spanners with a bundle of dynamic (approximate) maximum spanning forests [30, 33, 28] that could certify edge connectivities, they also showed fully dynamic algorithms for cut sparsification with worst-case update time of $\tilde{O}(\varepsilon^{-2})$ against an oblivious adversary.

These results were subsequently strengthened by Bernstein, van den Brand, Probst Gutenberg, Nanongkai, Saranurak, Sidford, and Sun [11] to either work against adaptive adversaries or have worst-case update time guarantees. At the core of these results is a black-box reduction that reduces fully dynamic sparsification problems to decremental sparsification problems on almost uniform degree expanders. A version of their framework serves as the basis of our dynamic algorithms. To handle probabilistic dependencies introduced by adaptive adversaries, Bernstein et al. developed a technique called *proactive* sampling, where an edge update incurs multiple re-samplings with increasing intervals. One downside is that their adaptive algorithms have a significant trade-off between update time and approximation factor, leading to at least polylogarithmic approximations for any subpolynomial update time.

2 Preliminaries

All log are base 2 unless otherwise specified. We denote $\log_e$ by $\ln$. When discussing a graph clear from context with n vertices and edge weight ratio bounded by W . We use the $\tilde{O}$ notation to hide $\text{polyloglog}(nW)$ factors for brevity (in runtimes only). We let $[n] \stackrel{\text{def}}{=} \{i \in \mathbb{N} \mid 1 \leq i \leq n\}$.

Vectors and Matrices

Vectors are denoted in lower-case boldface. $\mathbf{0}_d$ and $\mathbf{1}_d$ are the all-zeroes and all-ones vector respectively of dimension d . $\mathbf{e}_i$ denotes the i^{th} basis vector. $\mathbf{u} \circ \mathbf{v}$ denotes the entrywise product of $\mathbf{u}, \mathbf{v}$ of equal dimension.

Matrices are denoted in upper-case boldface. We refer to the i^{th} row and j^{th} column of matrix $\mathbf{M}$ by $\mathbf{M}_{i,:}$ and $\mathbf{M}_{:,j}$ respectively. We use $[\mathbf{v}]_i$ to index into the i^{th} coordinate of vector $\mathbf{v}$, and let $[\mathbf{M}]_{i,:} \stackrel{\text{def}}{=} \mathbf{M}_{i,:}$, $[\mathbf{M}]_{:,j} \stackrel{\text{def}}{=} \mathbf{M}_{:,j}$, and $[\mathbf{M}]_{ij} \stackrel{\text{def}}{=} \mathbf{M}_{ij}$ in contexts where $\mathbf{v}, \mathbf{M}$ have subscripts.

$\mathbf{I}_d$ is the $d \times d$ identity matrix. For $\mathbf{v} \in \mathbb{R}^d$, $\mathbf{diag}(\mathbf{v})$ denotes the associated diagonal $d \times d$ matrix. For linear subspace S of $\mathbb{R}^d$, $\dim(S)$ is its dimension and $\mathbf{P}_S$ is the orthogonal projection matrix onto S . We let $\ker(\mathbf{M})$, $\text{im}(\mathbf{M})$, and $\mathbf{M}^\dagger$ denote the kernel, image and pseudoinverse of $\mathbf{M}$. We denote the operator norm (largest singular value) of matrix $\mathbf{M}$ by $\|\mathbf{M}\|_{\text{op}}$. The number of nonzero entries of a matrix $\mathbf{M}$ (resp. vector $\mathbf{v}$) is denoted $\text{nnz}(\mathbf{M})$ (resp. $\text{nnz}(\mathbf{v})$), and the subset of indices with nonzero entries is $\text{supp}(\mathbf{M})$ (resp. $\text{supp}(\mathbf{v})$).

We use $\preceq$ to denote the Loewner partial order on $\mathbb{S}^d$, the symmetric $d \times d$ matrices. For $\mathbf{M} \in \mathbb{S}^d$ and $i \in [d]$, we let $\lambda_i(\mathbf{M})$ denote the i^{th} smallest eigenvalue of $\mathbf{M}$, so $\lambda_1(\mathbf{M}) \leq \lambda_2(\mathbf{M}) \leq \dots \leq \lambda_d(\mathbf{M})$. For positive semidefinite $\mathbf{A} \in \mathbb{S}^d$, we define the seminorm induced by $\mathbf{A}$ by $\|\mathbf{x}\|_{\mathbf{A}}^2 \stackrel{\text{def}}{=} \mathbf{x}^\top \mathbf{A} \mathbf{x}$.

Graphs

All graphs throughout this paper are assumed to be simple without loss of generality, as collapsing parallel multi-edges does not affect (undirected or directed) graph Laplacians. Moreover, a dynamic update of adding a parallel edge can be simulated by one edge deletion and one edge insertion with combined weight. We denote undirected weighted graphs without an arrow and directed weighted graphs with an arrow, i.e., $G = (V, E, \mathbf{w})$ is an undirected graph with vertices V , edges E , and weights $\mathbf{w} \in \mathbb{R}_{\geq 0}^E$, and $\vec{G}$ is a directed graph. A directed Eulerian graph is a directed graph where weighted in-degree equals weighted out-degree for every vertex. We refer to the vertex set and edge set of a graph G (resp. $\vec{G}$) by $V(G)$ and $E(G)$ (resp. $V(\vec{G})$ and $E(\vec{G})$). We associate a directed edge e from u to v with the tuple (u, v) , and an undirected edge with (u, v) and (v, u) interchangeably. We define $h(e) = u$ and $t(e) = v$ to be the head and tail of a directed edge $e = (u, v)$. We let $\text{rev}(\vec{G})$ denote the directed graph with all edge orientations reversed from $\vec{G}$, and $\text{und}(\vec{G})$ denote the undirected graph which removes orientations (both keeping the same weights).

We say H is a subgraph of G if the edges and vertices of H are subsets of the edges and vertices of G (with the same weights), denoting $H = G_F$ if $E(H) = F$, and defining the same notion for directed graphs. For $U \subseteq V$, we let $G[U]$ denote the induced subgraph of G on U (i.e., keeping all of the edges within U). When V is a set of vertices, we say $\{V_i\}_{i \in [I]}$ is a partition of V if $\bigcup_{i \in [I]} V_i = V$, and all V_i are disjoint. We say $\{G_j\}_{j \in [J]}$ are a family of edge-disjoint subgraphs of $G = (V, E, \mathbf{w})$ if all $E(G_j)$ are disjoint, and for all $j \in [J]$, $V(G_j) \subseteq V$, $E(G_j) \subseteq E$, and every edge weight in G_j is the same as its weight in G .

Graph matrices

For a graph with edges E and vertices V , we let $\mathbf{B} \in \{-1, 0, 1\}^{E \times V}$ be its edge-vertex transfer matrix, so that when $\vec{G}$ is directed and $e = (u, v)$, $\mathbf{B}_e$ is 2-sparse with $\mathbf{B}_{eu} = 1$, $\mathbf{B}_{ev} = -1$ (for undirected graphs, we fix an arbitrary consistent orientation). For $u, v \in V$, we define $\mathbf{b}_{(u,v)} \stackrel{\text{def}}{=} \mathbf{e}_u - \mathbf{e}_v$. When $\mathbf{B}$ is the transfer matrix associated with graph $G = (V, E, \mathbf{w})$ (resp. $\vec{G}$), we say $\mathbf{x}$ is a circulation in G (resp. $\vec{G}$) if $\mathbf{B}^\top \mathbf{x} = \mathbf{0}_V$; when G (resp. $\vec{G}$) is clear we simply say $\mathbf{x}$ is a circulation. We let $\mathbf{H}, \mathbf{T} \in \{0, 1\}^{E \times V}$ indicate the heads and tails of each edge, i.e., have one nonzero entry per row indicating the relevant head or tail vertex for each edge, respectively, so that $\mathbf{B} = \mathbf{H} - \mathbf{T}$. When clear from context that $\mathbf{w}$ are edge weights, we let $\mathbf{W} \stackrel{\text{def}}{=} \text{diag}(\mathbf{w})$. For undirected $G = (V, E, \mathbf{w})$ with transfer matrix $\mathbf{B}$, the Laplacian matrix of G is $\mathbf{L} \stackrel{\text{def}}{=} \mathbf{B}^\top \mathbf{W} \mathbf{B}$. For directed $\vec{G} = (V, E, \mathbf{w})$, the directed Laplacian matrix of $\vec{G}$ is $\vec{\mathbf{L}} \stackrel{\text{def}}{=} \mathbf{B}^\top \mathbf{W} \mathbf{H}$. To disambiguate, we use $\mathbf{L}_G, \mathbf{H}_G, \mathbf{T}_G, \mathbf{B}_G$, etc. to denote matrices associated with a graph G when convenient.

Note that $\vec{\mathbf{L}}^\top \mathbf{1}_V = \mathbf{0}_V$ for any directed Laplacian $\vec{\mathbf{L}}$. If $\vec{G}$ is Eulerian, then its directed Laplacian also satisfies $\vec{\mathbf{L}} \mathbf{1}_V = \mathbf{0}_V$ and $\mathbf{w}$ is a circulation in $\vec{G}$ (i.e., $\mathbf{B}^\top \mathbf{w} = \mathbf{0}_V$). Note that for a directed graph $\vec{G} = (V, E, \mathbf{w})$ and its corresponding undirected graph $G \stackrel{\text{def}}{=} \text{und}(\vec{G})$, the undirected Laplacian is $\mathbf{L}_G = \mathbf{B}^\top \mathbf{W} \mathbf{B}$, and the reversed directed Laplacian is $\vec{\mathbf{L}}_{\text{rev}(\vec{G})} = -\mathbf{B}^\top \mathbf{W} \mathbf{T}$.

We let Π_V denote the Laplacian of the unweighted complete graph on V , i.e., $\Pi_V \stackrel{\text{def}}{=} \mathbf{I}_V - \frac{1}{|V|} \mathbf{1}_V \mathbf{1}_V^\top$. Note that Π_V is the orthogonal projection on the subspace spanned by the vector that is 1 in the coordinates of V and 0 elsewhere.

For a subset of vertices $C \subseteq V$ and $F = V \setminus C$, the Schur complement of a directed Laplacian $\vec{\mathbf{L}}$ onto C is defined by $\text{Sc}(\vec{\mathbf{L}}, C) \stackrel{\text{def}}{=} \vec{\mathbf{L}}_{CC} - \vec{\mathbf{L}}_{CF} \vec{\mathbf{L}}_{FF}^\dagger \vec{\mathbf{L}}_{FC}$, which satisfies the block LU factorization

$$\vec{\mathbf{L}} = \begin{pmatrix} \vec{\mathbf{L}}_{FF} & \vec{\mathbf{L}}_{FC} \\ \vec{\mathbf{L}}_{CF} & \vec{\mathbf{L}}_{CC} \end{pmatrix} = \begin{pmatrix} \mathbf{I}_F & 0 \\ \vec{\mathbf{L}}_{CF} \vec{\mathbf{L}}_{FF}^\dagger & \mathbf{I}_C \end{pmatrix} \begin{pmatrix} \vec{\mathbf{L}}_{FF} & 0 \\ 0 & \text{Sc}(\vec{\mathbf{L}}, C) \end{pmatrix} \begin{pmatrix} \mathbf{I}_F & \vec{\mathbf{L}}_{FF}^\dagger \vec{\mathbf{L}}_{FC} \\ 0 & \mathbf{I}_C \end{pmatrix}. \quad (2)$$

A Schur complement of a directed graph Laplacian remains a directed graph Laplacian. We say that a directed graph $\vec{H}$ is the Schur complement of $\vec{G}$ onto a subset $C \subseteq V$, denoted by $\vec{H} = \text{Sc}(\vec{G}, C)$, if $\vec{\mathbf{L}}_{\vec{H}} = \text{Sc}(\vec{\mathbf{L}}, C)$. The Schur complement of an Eulerian graph is Eulerian.

Cut, Conductance and Expanders

Given a directed graph $\vec{G} = (V, E, \mathbf{w})$, for a subset $S \subseteq V$ of vertices, the volume of S is the same as $\text{vol}_G(S) \stackrel{\text{def}}{=} \sum_{v \in S} \sum_{e \in E(G): e \ni v} \mathbf{w}(e)$ the volume of S in the corresponding undirected graph $G = \text{und}(\vec{G})$. For another subset $T \subseteq V$, we let $E(S, T)$ be the set of edges satisfying that for $e \in E(S, T)$ it has $h(e) \in S$ and $t(e) \in T$ and denote by $\mathbf{w}(S, T) \stackrel{\text{def}}{=} \sum_{e \in E(S, T)} \mathbf{w}(e)$ the total weight of these edges. We say that a subgraph $\vec{C}$, often referred to as a subset of edges, is a directed cut (di-cut) of $\vec{G}$ if there exists some subset $S \subseteq V$ satisfying $S, V \setminus S \neq \emptyset$ such that $E(\vec{C}) = E(S, V \setminus S)$. The corresponding (undirected) cut $C = \text{und}(\vec{C})$ of $\vec{C}$ is defined by $E(C) = E_G(S, V \setminus S) = E(S, V \setminus S) \cup E(V \setminus S, S)$ the set of cut edges in G .

We say that the undirected conductance of subset S is

$$\Phi_G(S) = \frac{\mathbf{w}_G(S, V \setminus S)}{\min \{\text{Vol}(S), \text{Vol}(V \setminus S)\}},$$

where we note that $\mathbf{w}_G(S, V \setminus S) = \mathbf{w}(S, V \setminus S) + \mathbf{w}(V \setminus S, S)$. The conductance of an undirected graph G , denoted by $\phi(G)$, is the minimum conductance $\phi_G(S)$ of any cut $(S, V \setminus S)$.

► **Definition 9.** For any $\phi \in (0, 1)$, we say that an undirected graph G is a ϕ -expander if $\Phi(G) \geq \phi$.

3 A Fully Dynamic Directed Spectral Sparsifier

In this section, we give an overview of our main result, Theorem 3 for dynamically maintaining directed spectral sparsifiers under the notion of degree balance preserving directed spectral approximation (Definition 2).

For directed spectral sparsification, a major challenge compared to undirected spectral sparsification is the degree balance preservation requirement. Sampling edges independently, as in undirected spectral sparsification algorithms [46], may create degree imbalances. Recent approaches in the static setting instead use independent sampling of graph structures that inherently preserve degree balances. Chu et al. [16] used short-cycle sampling while Jambulapati et al. [32] employed electrical circulation sampling. These structures are particularly difficult to maintain in a dynamic setting and the recourse could grow quickly over $\Omega(\log n)$ levels of half-sparsifications.

To avoid these issues, we instead adopt the approach of Cohen et al. [20], where the sparsification algorithm performs independent edge sampling on expanders and computes a “patching” graph to fix degree imbalances caused by the sampling. Cohen et al. made the key observation that sampling a directed edge with probability inversely proportional to both the

in-degree of its tail and the out-degree of its head in an “expander” (i.e., a directed graph whose corresponding undirected graph is an expander) effectively controls spectral error and degree imbalance. They then showed that a small patching of size $O(n)$ can be computed for such an expander by greedily matching remaining imbalanced degrees.

We show in the full version that a patching similar to the greedy patching of [20] can be implicitly maintained by a tree data structure that allows $\tilde{O}(1)$ time querying of any edge. We call this an *external* patching since it introduces directed edges not originally present in the graph. To avoid having extra edges in a sparsifier, we introduce another patching algorithm that slightly adjusts the edge weights of the sampled subgraph using electrical flow. We call this an *internal* patching algorithm. For our dynamic algorithm, we only update the demand vector associated with the degree imbalance and compute an electric flow when querying the entire graph. We refer readers to the full version for more details.

A significant problem with both patching approaches is that the patching edges cannot be maintained explicitly. For external patching, modifying the degree of a vertex whose interval appears early in the ordering changes almost all edge weights in the patching. It can be worse for internal patching: the entire electrical flow is changed by a single update. To solve this problem, we observe that by adding auxiliary vertices, we can effectively patch degree imbalance using just a single directed star graph with one extra vertex. This approach allows us to maintain the spectral sparsifier explicitly and efficiently, as a single degree change only requires adjusting one edge on the degree imbalance. We remark that there is no tangible drawback for having a few auxiliary vertices in a sparsifier. For Eulerian graphs specifically, sparsifiers constructed this way still serve as effective preconditioners for solving Eulerian systems (see Lemma 13 in Section 3.1). There are also applications of undirected sparsifiers with auxiliary vertices [39, 26].

3.1 Properties of degree balance preserving directed spectral sparsifiers

In this section, we provide some useful properties of our degree balance preserving directed spectral sparsification. Some of these properties are crucial for applying the dynamic sparsification framework of [11]. Proofs of these statements can be found in the full version.

We state first the following equivalent definition of Definition 2 using the degree preserving condition, justifying our naming for Definition 2.

► **Lemma 10.** $\vec{G}'$ is an ε -degree balance preserving directed spectral approximation of $\vec{G}$ (i.e., satisfying (1)) if and only if $\vec{G}'$ is degree balance preserving w.r.t. $\vec{G}$ and

$$\left\| \mathbf{L}_G^{\frac{1}{2}} \left(\vec{\mathbf{L}}_{\vec{G}} - \vec{\mathbf{L}}_{\vec{H}} \right) \mathbf{L}_G^{\frac{1}{2}} \right\|_{\text{op}} \leq \varepsilon. \quad (3)$$

Recall the *union property* of graph sparsification problems. Let $\mathcal{H}(\vec{G}, \varepsilon)$ be the set of ε -approximations of $\vec{G}$ under some definition of approximation for weighted directed (or undirected) graph. Then, we say that $\mathcal{H}$ satisfies union property if for any $G = \bigcup_{i=1}^k s_i \cdot G_i$ with $s_1, \dots, s_k \in \mathbb{R}_{\geq 0}$ it satisfies that for any $\vec{H}_i \in \mathcal{H}(\vec{G}_i, \varepsilon)$, we have

$$\bigcup_{i=1}^k s_i \cdot \vec{H}_i \in \mathcal{H}(\vec{G}, \varepsilon).$$

Our notion of directed spectral sparsification Definition 2 naturally satisfies the union property.

► **Lemma 11** (Union property). *Suppose directed graph $\vec{G} = \bigcup_{i=1}^k s_i \cdot \vec{G}_i$ for some k and that $s_1, \dots, s_k \in \mathbb{R}_{\geq 0}$. Then, suppose $\vec{G}'_i$ is an ε -degree balance preserving directed spectral approximation to $\vec{G}_i$ for every $i \in [k]$, it follows that $\bigcup_{i=1}^k s_i \vec{G}'_i$ is an ε -degree balance preserving directed spectral approximation to $\vec{G}$.*

We state also the *contraction property* where contraction of vertices does not affect the quality of our degree-balance preserving approximation.

► **Lemma 12** (Contraction property). *For a directed graph $\vec{G} = (V, E, \mathbf{w})$, suppose $\vec{H}$ is an ε -degree balance preserving directed spectral approximation to $\vec{G}$. Then, for any subset $W \subseteq V$, let $\vec{G}', \vec{H}'$ be the respective directed graphs resulting from contracting W , then $\vec{H}'$ is an ε -degree balance preserving directed spectral approximation to $\vec{G}'$.*

We also demonstrate that having extra vertices in a degree balance preserving sparsifier does not affect most potential applications of it. In particular, Lemma 13 states that as long as the Schur complement of a graph $\vec{H}$ is a good degree balance preserving sparsifier of an Eulerian graph $\vec{G}$, then $\vec{\mathbf{L}}_{\vec{H}}^\dagger$ serves as a good preconditioner of $\vec{\mathbf{L}}_{\vec{G}}$.

► **Lemma 13.** *Let $\vec{G} = (V, E, \mathbf{w})$ and $\vec{H} = (V \cup X, E_{\vec{H}}, \mathbf{w}_{\vec{H}})$ be directed graphs such that the Schur complement $\text{Sc}(\vec{H}, V)$ is a ε -degree balance preserving approximation of $\vec{G}$ for $\varepsilon \in (0, \frac{1}{4})$. If $\vec{G}$ is Eulerian, then*

$$\left\| \mathbf{P}_V - \mathbf{P}_V \vec{\mathbf{L}}_{\vec{H}}^\dagger \vec{\mathbf{L}}_{\vec{G}} \right\|_{\mathbf{L}_G \rightarrow \mathbf{L}_G} \leq 8\varepsilon.$$

3.2 Bipartite lifts

We invoke a helper result which we use to ensure degree preserving properties, a strictly stronger property of degree balance preserving properties, of our algorithms by working with bipartite lifts. This primitive is from Section 9.1 of [32]. Given a directed graph $\vec{G} = (V, E, \mathbf{w})$, we let the directed graph $\vec{G}^\uparrow \stackrel{\text{def}}{=} \text{blift}(\vec{G})$ be its bipartite lift, which is defined so that $V_{\vec{G}^\uparrow} = V \cup \bar{V}$ where $\bar{V}$ is a copy of V , and $E_{\vec{G}^\uparrow} = \{f = (u, \bar{v}) \mid (u, v) \in E\}$ with $\mathbf{w}_{(u, \bar{v})} = \mathbf{w}_{(u, v)}$. Notice that our definition gives a canonical bijection between $E_{\vec{G}^\uparrow}$ and $E_{\vec{G}}$.

► **Lemma 14.** *Let $\vec{G} = (V, E, \mathbf{w})$ be a directed graph and let its bipartite lift be $\vec{G}^\uparrow \stackrel{\text{def}}{=} (V \cup V', E^\uparrow, \mathbf{w}) = \text{blift}(\vec{G})$, with $G \stackrel{\text{def}}{=} \text{und}(\vec{G})$ and $G^\uparrow \stackrel{\text{def}}{=} \text{und}(\vec{G}^\uparrow)$. Suppose that for some $\varepsilon > 0$, $\mathbf{w}' \in \mathbb{R}_{>0}^E$ satisfies*

$$\mathbf{B}_{\vec{G}^\uparrow}^\top \mathbf{w}' = \mathbf{B}_{\vec{G}^\uparrow}^\top \mathbf{w}, \quad \left\| \mathbf{L}_{G^\uparrow}^{\frac{1}{2}} \mathbf{B}_{\vec{G}^\uparrow}^\top (\mathbf{W}' - \mathbf{W}) \mathbf{H}_{\vec{G}^\uparrow} \mathbf{L}_{G^\uparrow}^{\frac{1}{2}} \right\|_{\text{op}} \leq \varepsilon.$$

Then, letting $|\mathbf{B}_{\vec{G}}|$ apply the absolute value entry-wise,

$$\mathbf{B}_{\vec{G}}^\top \mathbf{w}' = \mathbf{B}_{\vec{G}}^\top \mathbf{w}, \quad |\mathbf{B}_{\vec{G}}|^\top \mathbf{w}' = |\mathbf{B}_{\vec{G}}|^\top \mathbf{w}, \quad \left\| \mathbf{L}_G^{\frac{1}{2}} \mathbf{B}_{\vec{G}}^\top (\mathbf{W}' - \mathbf{W}) \mathbf{H}_{\vec{G}} \mathbf{L}_G^{\frac{1}{2}} \right\|_{\text{op}} \leq \varepsilon.$$

For the rest of this section, we almost exclusively considers such directed bipartite graphs.

Lemma 15 states that adding extra vertices and taking the Schur complement onto the original vertices under bipartite lift does not affect approximation guarantees on the unlifted graph.

► **Lemma 15.** *Let $\vec{G} = (V, E, \mathbf{w})$ be a directed graph and let its bipartite lift be $\vec{G}^\uparrow \stackrel{\text{def}}{=} (V \cup V', E^\uparrow, \mathbf{w}) = \text{blift}(\vec{G})$, with $G \stackrel{\text{def}}{=} \text{und}(\vec{G})$ and $G^\uparrow \stackrel{\text{def}}{=} \text{und}(\vec{G}^\uparrow)$. Suppose $\vec{H} = (V \cup V' \cup X, E_{\vec{H}}, \mathbf{w}_{\vec{H}})$ is a directed graph such that $\text{Sc}(\vec{H}, V \cup V')$ is an ε -degree preserving spectral approximation of $\vec{G}^\uparrow$. Let $\vec{H}^\downarrow$ be the resulting directed graph of $\vec{H}$ with each pair of corresponding vertices $\{v, v'\}$ contracted. Then, $\text{Sc}(\vec{H}^\downarrow, V)$ is an ε -degree preserving spectral approximation of $\vec{G}$.*

3.3 Reduction to decremental sparsifier on expanders

In this section, we recall a fully dynamic expander decomposition algorithm by [11] using expander pruning from [45].

We first state the definition of expander decompositions. Note that for the purpose of this paper, we assume that the subgraphs in the decomposition contain the *entire set of edges*.

► **Definition 16** (Expander decomposition). *We call $(\{G_i\}_{i \in [I]})$ a (ϕ, J) -expander decomposition if $\{G_i\}_{i \in [I]}$ are edge-disjoint subgraphs of $G = (V, E, \mathbf{w})$ satisfying $\bigcup_i G_i = G$, and the following hold.*

1. Bounded weight ratio: *For all $i \in [I]$, $\frac{\max_{e \in E(G_i)} \mathbf{w}_e}{\min_{e \in E(G_i)} \mathbf{w}_e} \leq 2$.*
2. Conductance: *For all $i \in [I]$, $\Phi(G_i) \geq \phi$.*
3. Vertex coverage: *There is a J -partition $\bigcup_{j \in [J]} S_j = [I]$ such that every vertex $v \in V(G)$ appears at most once in the subgraphs of each S_j .*

When stating our results, we replace the use of the static expander decomposition algorithm from [45] by the state-of-the-art algorithm in [2].

► **Lemma 17** (Expander pruning). *Let $G = (V, E, \mathbf{w})$ be a ϕ -expander with $\frac{\max_{e \in E} \mathbf{w}_e}{\min_{e \in E} \mathbf{w}_e} \leq 2$. There is a deterministic algorithm with access to adjacency lists of G such that given an online sequence of $k \leq \frac{\phi m}{10}$ edges deletions in G , can maintain a pruned set $P \subset V$ satisfying the following properties. Let G_i, P_i be the graph and set P after i edge deletions. We have for all i*

1. $P_0 = \emptyset, P_i \subseteq P_{i+1}$,
2. $\text{vol}_{G_i}(P_i) \leq \frac{16i}{\phi} \min_{e \in E} \mathbf{w}_e$,
3. $G_i[V \setminus P_i]$ is a $\frac{1}{12}\phi$ -expander,
4. $|E(P_i, V \setminus P_i)| \leq 4i$.

The amortized update time is $O(\phi^{-2} \log m)$.

► **Proposition 18** (Dynamic expander decomposition). *Given a weighted undirected dynamic graph G with edge bounded edge weights ratio $\frac{\max_{e \in E} \mathbf{w}_e}{\min_{e \in E} \mathbf{w}_e} \leq W$ at all time, there exists a dynamic algorithm against an adaptive adversary that preprocess in $O(m \log^7 n)$ time, maintains with high probability a $(C_{\text{DED}} \log^{-2}(n), (\log(W) + 3)(2 \log n + 3) \log n)$ -expander decomposition over all polynomially bounded number of updates of G with high probability for some constant C_{DED} . The algorithm supports both edge deletions and insertions with $O(\log^2 n)$ amortized recourse and $O(\log^7 n)$ amortized time.*

After each update, the output consists of a list of potential changes to the decomposition: (i) edge deletions to some subgraph in the decomposition, (ii) removing some subgraph from the decomposition, and (iii) new subgraph added to the decomposition.

Crucially, the types of updates in Proposition 18 entails that any expander in the dynamic decomposition only receives decremental update before it becomes obsolete.

We now recall the reduction to decremental algorithm on expanders by [11] using dynamic expander decomposition algorithm from above. The version we present here is an informal statement with a slight generalization: we remove the perturbation requirement and allows for directed graphs. See full version for more details.

► **Lemma 19** (Informal blackbox reduction with dynamic expander decomposition). *Assume $\mathcal{H}$ is a graph sparsification problem that satisfies the union property, and there is a decremental algorithm $\mathcal{A}$ for $\mathcal{H}$ on (directed) graphs with edge weights ratio at most 2 and satisfying that their corresponding undirected graph is an expander.*

■ **Algorithm 1** PATCHINGSTAR($\vec{G} = (V, E, \mathbf{w}), \mathbf{w}', \xi$).

-
- 1 **Input:** $\vec{G} = (V, E, \mathbf{w})$ a simple weighted directed bipartite graph with bipartition $V = C \cup R, E \subseteq C \times R$ and edge-vertex transfer matrix $\mathbf{B} = \mathbf{H} - \mathbf{T}$, $\mathbf{w}'$ a reweighting of the edges, $\xi \in (0, 1]$
 - 2 Let $\vec{H}$ be $\vec{G}$ but with one extra vertex x . $\mathbf{d}_1 = \mathbf{H}^\top (\mathbf{w} - \xi \mathbf{w}')$, $\mathbf{d}_2 = \mathbf{T}^\top (\mathbf{w} - \xi \mathbf{w}')$
 - 3 For each $v \in C$ with $[\mathbf{d}_1]_v \neq 0$, $E_{\vec{H}} \leftarrow E_{\vec{H}} \cup (v, x)$, and $[\mathbf{w}_{\vec{H}}]_{(v,x)} \leftarrow [\mathbf{d}_1]_v$
 - 4 For each $v \in R$ with $[\mathbf{d}_2]_v \neq 0$, $E_{\vec{H}} \leftarrow E_{\vec{H}} \cup (x, v)$, and $[\mathbf{w}_{\vec{H}}]_{(x,v)} \leftarrow [\mathbf{d}_2]_v$
 - 5 **return** $\vec{H} = (V \cup \{x\}, E_{\vec{H}}, \mathbf{w}_{\vec{H}})$
-

Then, there exists a fully dynamic algorithm $\mathcal{B}$ for $\mathcal{H}$ on weighted (directed) graphs such that given a dynamic weighted simple directed graph $\vec{G}$ maintains an ε -approximate sparsifier of essentially the same size with essentially the same amortized update time as $\mathcal{A}$.

3.4 Patching by directed star graphs

To fix the degree imbalance, we consider patching using directed star graphs. The static algorithm PATCHINGSTAR (Algorithm 1) can be trivially made dynamic with deterministic updates in $O(1)$ worst-case update time and recourse. This dynamic algorithm can also maintain the guarantees in Lemma 20.

► **Lemma 20** (Star patching). *Given $\vec{G}$, $\mathbf{w}'$ as in the input conditions and ξ satisfying that $\xi[\mathbf{d}']^{(o)} \leq \mathbf{d}^{(o)}$ and $\xi[\mathbf{d}']^{(i)} \leq \mathbf{d}^{(i)}$ for $[\mathbf{d}']^{(i)}, [\mathbf{d}']^{(o)}$ the in and out degrees of $\vec{G}' = (V, E, \mathbf{w}')$, PATCHINGSTAR (Algorithm 1) returns in time $O(m)$ a simple weighted directed graph $\vec{H}$ such that*

1. $\mathbf{d}^{(o)} = [\mathbf{d}_{\vec{H}}^{(o)}]_V$, $\mathbf{d}^{(i)} = [\mathbf{d}_{\vec{H}}^{(i)}]_V$, and for the Schur complement $\vec{S} = \text{Sc}(\vec{H}, V)$, $\mathbf{d}_{\vec{S}}^{(o)} = \mathbf{d}_{\vec{H}}^{(o)}$, $\mathbf{d}_{\vec{S}}^{(i)} = \mathbf{d}_{\vec{H}}^{(i)}$,
- 2.

$$\begin{aligned} & \left\| [\mathbf{D}^{(i)}]^{\frac{1}{2}} \left(\text{Sc}(\vec{L}_{\vec{H}}, V) - \vec{L}_{\vec{G}'} \right) [\mathbf{D}^{(o)}]^{\frac{1}{2}} \right\|_{\text{op}} \\ & \leq (1 - \xi) + \max \left\{ \left\| \mathbf{1}_R - [\mathbf{D}^{(i)}]^\dagger [\mathbf{d}']^{(i)} \right\|_\infty, \left\| \mathbf{1}_C - [\mathbf{D}^{(o)}]^\dagger [\mathbf{d}']^{(o)} \right\|_\infty \right\}, \end{aligned}$$

3. $m_{\vec{H}} \leq \text{nnz}(\mathbf{w}') + n$ and $|V_{\vec{H}}| = n + 1$.

3.5 A simple decremental algorithm on expanders

Finally, we present our main ingredient for a fully dynamic spectral sparsifier: a decremental algorithm for maintaining a degree preserving directed spectral sparsifier for an expander.

► **Lemma 21.** *Suppose $\vec{G} = (C \cup R, E, \mathbf{w})$ is a weighted directed bipartite graph with edges $E \subseteq C \times R$ and weight ratio $\frac{\max_{e \in E} \mathbf{w}_e}{\min_{e \in E} \mathbf{w}_e} \leq 2$ undergoing edge deletions only. There is a dynamic algorithm that preprocesses in time $O(m)$, maintains explicitly a directed graph $\vec{H}$ on vertices $C \cup R \cup \{x\}$ and $m_{\vec{H}} = O(\varepsilon^{-2} \phi^{-4} N \log n)$, where N is the number of non-trivial vertices currently. The algorithm has worst-case recourse and update time $O(\varepsilon^{-2} \phi^{-8} \log^2 n)$. Furthermore, with high probability whenever $G = \text{und}(\vec{G})$ becomes a ϕ -expander, $\text{Sc}(\vec{H}, C \cup R)$ is an ε -degree preserving directed spectral sparsifier of $\vec{G}$.*

The following Lemma 22 is the key lemma from [20] that enables our results. In light of our static algorithm, our dynamic algorithm maintains a reweighted subgraph $\vec{G}'$ satisfying the approximation guarantees in (4) for its weighted adjacency matrix $\mathbf{A}' = \mathbf{T}^\top \mathbf{W}' \mathbf{H}$ with $\mathbf{R} = \mathbf{D}^{(i)}$ and $\mathbf{C} = \mathbf{D}^{(o)}$. For brevity we say that $\vec{G}'$ is an ε -degree approximation of $\vec{G}$.

► **Lemma 22.** *Let $\mathbf{A} \in \mathbb{R}_{\geq 0}^{n_1 \times n_2}$ be a matrix with $m \stackrel{\text{def}}{=} \text{nnz}(\mathbf{A})$, $n \stackrel{\text{def}}{=} (n_1 + n_2)$ and parameters $\varepsilon, \delta \in (0, 1)$. Let the row and column sums be $\mathbf{r} = \mathbf{A} \mathbf{1}_{d_2}$ and $\mathbf{c} = \mathbf{A}^\top \mathbf{1}_{d_1}$ with $\mathbf{R} \stackrel{\text{def}}{=} \text{diag}(\mathbf{r})$ and $\mathbf{C} \stackrel{\text{def}}{=} \text{diag}(\mathbf{c})$. Consider a random matrix $\tilde{\mathbf{A}} \in \mathbb{R}_{\geq 0}^{n_1 \times n_2}$ that independently for each nonzero $\mathbf{A}_{ij}$*

$$\tilde{\mathbf{A}}_{ij} = \begin{cases} \frac{1}{p_{ij}} \mathbf{A}_{ij} & \text{with probability } p_{ij} \geq \min \left\{ 1, \rho \mathbf{A}_{ij} \left(\frac{1}{\sqrt{\mathbf{r}_i \mathbf{c}_j}} \right) \right\} \\ 0 & \text{otherwise,} \end{cases}$$

where $\rho = C_{\text{SS}} \cdot \varepsilon^{-2} \log \frac{3n}{\delta}$. Then, it satisfies that with probability $\geq 1 - \delta$, the following holds

$$\left\| \mathbf{R}^{\frac{1}{2}} (\tilde{\mathbf{A}} - \mathbf{A}) \mathbf{C}^{\frac{1}{2}} \right\|_{\text{op}} \leq \varepsilon, \quad \left\| \mathbf{R}^\dagger (\tilde{\mathbf{A}} - \mathbf{A}) \mathbf{1}_{n_2} \right\|_\infty \leq \varepsilon, \quad \left\| \mathbf{C}^\dagger (\tilde{\mathbf{A}} - \mathbf{A})^\top \mathbf{1}_{n_1} \right\|_\infty \leq \varepsilon. \quad (4)$$

■ **Algorithm 2** DEGREE SPARSIFY PREPROCESS($\vec{G} = (V, E, \mathbf{w}), \varepsilon, \delta$).

-
- 1 **Input:** $\vec{G} = (V, E, \mathbf{w})$ a simple weighted directed graph with edge weights ratio $\frac{\max_{e \in E} \mathbf{w}_e}{L} \leq 2$ for all $e \in E$ where we set $L \stackrel{\text{def}}{=} \min_{e \in E} \mathbf{w}_e$ the initial smallest edge weight, $\varepsilon, \delta \in (0, 1)$
 - 2 $\rho \stackrel{\text{def}}{=} C_{\text{DS}} \cdot \varepsilon^{-2} \log(8n)$
 - 3 Initialize $\mathbf{w}' \leftarrow \mathbf{0}_E$ and compute $\mathbf{d}^{(i)}, \mathbf{d}^{(o)}$ the in and out-degree vectors of $\vec{G}$
 - 4 **foreach** $v \in V$ **do**
 - 5 $F_v \leftarrow \text{SUBSETSAMPLE}(E_v, \frac{2L\rho}{\mathbf{d}_v})$
 - 6 $\mathbf{w}'_e \leftarrow \frac{1}{p_e} \mathbf{w}_e$ for each $e \in F_v$, where $p_e = 1$ if either $\mathbf{d}_{h(e)} \leq 2L\rho$ or $\mathbf{d}_{t(e)} \leq 2L\rho$
 and $p_e = 4L^2\rho^2 \frac{(\mathbf{d}_{h(e)} + \mathbf{d}_{t(e)} - 1)}{\mathbf{d}_{h(e)} \mathbf{d}_{h(e)}}$ otherwise
-

■ **Algorithm 3** DEGREE SPARSIFY DELETE(e).

-
- 1 Update $[\mathbf{d}^{(i)}]_v, [\mathbf{d}^{(o)}]_v$ for each $v \in e$
 - 2 **foreach** $v \in e$ **such that** $v \in V_S$ **do**
 - 3 $F_v \leftarrow \text{SUBSETSAMPLE}(E_v, \frac{2L\rho}{\mathbf{d}_v})$
 - 4 $\mathbf{w}'_e \leftarrow \frac{1}{p_e} \mathbf{w}_e$ for each $e \in F_v$, where $p_e = 1$ if either $\mathbf{d}_{h(e)} \leq 2L\rho$ or $\mathbf{d}_{t(e)} \leq 2L\rho$
 and $p_e = 4L^2\rho^2 \frac{(\mathbf{d}_{h(e)} + \mathbf{d}_{t(e)} - 1)}{\mathbf{d}_{h(e)} \mathbf{d}_{h(e)}}$ otherwise
 - 5 Update $\mathbf{d}^{(i)}, \mathbf{d}^{(o)}$
-

Similar to the algorithm in [11], Algorithms 2, 3 sample for each vertex a subset of its incident edges using a fast uniform subset sampling algorithm [35, 22, 12]. An edge $e = (u, v)$ is present in the sparsifier as long as one of u, v has it sampled.

► **Lemma 23.** *Under the same assumptions on $\vec{G}, \varepsilon, \phi$ as in Lemma 21, there is an algorithm that given an oblivious online sequence of edge deletions, preprocesses in time $O(m)$, maintains explicitly a reweighted directed subgraph $\vec{G}'$ that is an ε -degree approximation of $\vec{G}$ with $\text{nnz}(\mathbf{w}') = O(\varepsilon^2 N \log n)$ where N is the number of non-trivial vertices currently. The algorithm has worst-case recourse $O(\varepsilon^{-2} \log n)$ and worst-case update time $O(\varepsilon^{-2} \log^2 n)$.*

Our main theorem of this section, Theorem 3, follows by combining our decremental sparsifier on expander in Lemma 21 and the dynamic expander decomposition framework in Lemma 19.

3.6 Degree preserving directed spectral sparsification

In this section, we show to extend the degree-balance preserving guarantee from the algorithms above to exact degree preserving, i.e., the algorithm maintains a sparsifier with exact same weighted in and out degrees as the input directed graph. Even though our algorithms are degree preserving on bipartite lifted graphs, the patching schemes, except internal patching, may introduce self-loops when “unlifting” the sparsifier back to the original set of vertices. Eliminating the center of a star creates a biclique on all of its adjacent vertices. If there was an edge (v, x) and (x, v') for center x , then the biclique contains the degenerative edge of (v, v') , making it not degree preserving.

To avoid self-loops, we instead consider using multiple stars. We ensure for each star that the incident edges of the center do not contain any pair of edges of the form (v, x) , (x, v') . For vertex sets V, V' , and the complete set of bipartite edges $E = V \times V'$, we devise a decomposition. Firstly, we label the vertices of V by $[n]$ where $n = |V|$ and denote by l its labeling. For each v' that corresponds to v , we assign it the same label. Then, we defined for each integer $i \in [1, \lceil \log_2(n) \rceil]$ the edge set E_i by

$$(u, v') \in E_i \quad \text{if} \quad \left\lfloor \frac{l(u)}{2^{i-1}} \right\rfloor \neq \left\lfloor \frac{l(v')}{2^{i-1}} \right\rfloor \quad \text{and} \quad \left\lfloor \frac{l(u)}{2^i} \right\rfloor = \left\lfloor \frac{l(v')}{2^i} \right\rfloor.$$

We then get the following *disjoint* decomposition of E ,

$$E = \{(v, v') | v \in V\} \cup \left(\bigcup_{i=1}^{\lceil \log_2(n) \rceil} E_i \right).$$

To produce a degree preserving directed spectral sparsifier, we first decompose the bipartite lift $\vec{G}^\uparrow = (V \cup V', F, \mathbf{w})$ into $\lceil \log_2(n) \rceil$ edges disjoint subgraphs where each $\vec{G}_i$ only contains the edges $F \cap E_i$. Our dynamic algorithm then proceeds the same as before, with one copy of the algorithm on each $\vec{G}_i$. Now, suppose the algorithm receives an update (insertion or deletion) with edge $e = (u, v')$, we first check for which E_i the edge e belongs to. The update is then propagated to the copy on $\vec{G}_i^\uparrow$ and processed. Our final sparsifier $\vec{H}$ is a union of our degree preserving sparsifiers $\vec{H}_i^\uparrow$ of each $\vec{G}_i^\uparrow$ unlified back to the original set of vertices. The correctness of these operations are guaranteed by Lemmas 11 and 15. We summarize our conclusion above into the following informal statement.

► **Theorem 24** (Theorem 3, degree preserving). *Given a directed graph $\vec{G}$ on vertices V and with polynomially bounded edge weights, we can maintain explicitly a graph $\vec{H}$ on vertices $V \cup X$ such that the Schur complement $\text{Sc}(\vec{H}, V)$ onto the original vertices is a ε -degree preserving directed spectral sparsifier of $\vec{G}$. $\vec{H}$ has size $\tilde{O}(\varepsilon^{-2}n)$ and satisfies $|X| = \tilde{O}(n)$. The algorithm has amortized recourse and update time $\tilde{O}(\varepsilon^{-2})$ per edge insertion or deletion and works against an oblivious adversary.*

4 A Fully Dynamic Balanced Directed Cut Sparsifier

In this section we give an overview of our fully dynamic algorithms for maintaining a β -balanced directed cut sparsifier under edges updates.

In the static setting, Cen, Cheng, Panigrahi, and Sun [13] initiated the study of β -balanced directed cut sparsification. Cen et al. [13] achieved $O(\varepsilon^{-2}\beta n \log n)$ sparsity by independent edge sampling with probability proportional to inverses of undirected *edge strength* defined originally by Benczúr and Karger [9] for undirected cut sparsification. As it stands, there is currently no known algorithm for fast maintenance of edge strength or fast certification of large edge strength under edge updates. We instead turn our attention to *undirected edge connectivity* that is proven to be useful for undirected cut sparsification in the work of [27]. We show statically that a β -balanced dicut sparsifier can also be obtained by sampling using undirected edge connectivity in Theorem 6.

► **Definition 25 (Edge connectivity).** *For any pair of vertices $u, v \in V$ in a weighted undirected graph $G = (V, E, \mathbf{w})$, the edge connectivity $\mathbf{k}_{uv}$ between u, v is defined as the minimum value of a cut that separates them. The connectivity of an edge $e = (u, v)$ is defined by $\mathbf{k}_e = \mathbf{k}_{uv}$.*

If an undirected graph is an expander, its edge connectivities can be certified by the degrees.

► **Lemma 26.** *For ϕ -expander $G = (V, E, \mathbf{w})$ it satisfies that for ever edge $e \in E$, the edge connectivity $\mathbf{k}_e \geq \phi \cdot \min_{v \ni e} \mathbf{d}_v$.*

One can then leverage expander decompositions to devise a fast algorithm for directed cut sparsifications. Such an algorithm lends itself nicely to the dynamic setting by utilizing the dynamic expander decomposition framework presented in Section 3.3.

► **Theorem 27.** *Given a directed graph with polynomially bounded edge weights, we can maintain explicitly a (β, ε) -balanced directed cut sparsifier of size $\tilde{O}(\varepsilon^{-2}\beta n)$ with amortized update time $\tilde{O}(\varepsilon^{-2}\beta)$ per edge insertion or deletion against an oblivious adversary.*

Our de-amortized result in Theorem 5 uses instead the dynamic framework developed by [1]. While it is possible to directly apply the black-box reduction developed in [11] for this task, their framework introduces significant runtime overhead since it is geared towards reduction of dynamic algorithms for adaptive adversary. The algorithm by [1] dynamically maintains bundles of approximate maximum spanning forests (called α -MSFs), that can certify edges with small undirected edge connectivities. We refer readers to the full version for more details on this approach.

Before we present our sparsification algorithms, we state a useful property of β -balanced dicut approximations. In the full version, we give these properties for a more general notion of cut approximation on digraphs.

► **Lemma 28 (Union property).** *Suppose directed graph $\vec{G} = \bigcup_{i=1}^k s_i \cdot \vec{G}_i$ for some k and that $s_1, \dots, s_k \in \mathbb{R}_{\geq 0}$. Then, suppose $\vec{H}_i$ is a (β, ε) -balanced dicut approximation to $\vec{G}_i$ for every $i \in [k]$, it follows that $\bigcup_{i=1}^k s_i \vec{H}_i$ is a (β, ε) -balanced dicut approximation to $\vec{G}$.*

4.1 Balanced directed cut sparsifiers via edge connectivity

Our static algorithm samples directed edges based on the reciprocals of their undirected edge connectivity measures. Proposition 29 quantifies the number of edges in a sparsifier.

► **Proposition 29.** *For a connected undirected graph $G = (V, E, \mathbf{w})$ with edge connectivity $\mathbf{k}$,*

$$\sum_{e \in E} \frac{\mathbf{w}_e}{\mathbf{k}_e} = n - 1. \quad (5)$$

157:16 Fully Dynamic Spectral and Cut Sparsifiers for Directed Graphs

We say that $\tilde{\mathbf{k}}$ is a ℓ -stretch edge connectivity estimation of G if $\tilde{\mathbf{k}}_e \leq \mathbf{k}_e$ for all e and

$$\sum_{e \in E} \frac{\mathbf{w}_e}{\tilde{\mathbf{k}}_e} \leq \ell \cdot \sum_{e \in E} \frac{\mathbf{w}_e}{\mathbf{k}_e}.$$

► **Definition 30.** An edge is k -heavy if its connectivity is at least k ; otherwise, it is said to be k -light. The k -projection of a cut is the set of k -heavy edges in it.

The key to our analysis is the following cut counting statement in Theorem 31 that is a generalization of Theorem 2.3 of [27] to weighted graphs. Crucially, Theorem 31 allows us to bound the number of distinct k -projections polynomially, allowing us to obtain high probability concentration.

► **Theorem 31** (Generalized cut projection counts, [27]). For a weighted undirected graph G , any $k \geq \lambda$ and any $\alpha \geq 1$, the number of distinct k -projections in cuts of value at most αk in G is at most $n^{C_{\text{cpc}} \alpha}$, where λ is the minimum value of a cut in G and $C_{\text{cpc}} \geq 2$ is a constant.

Similar to [27], we partition the edges in $\vec{G}$ according to their edge connectivities in G into sets $F_0, F_1, \dots, F_K$, where $K = \left\lceil \log \frac{\max_{e \in E} \mathbf{k}_e}{\lambda} \right\rceil$ where $\lambda = \min_{e \in E(G)} \mathbf{k}_e$ is the min-cut value of G and

$$F_i = \{e \in E : 2^i \lambda \leq \mathbf{k}_e \leq (2^{i+1} - 1) \lambda\}. \quad (6)$$

For a dicut $\vec{C}$ in $\vec{G}$ induced by a partition of the vertices $(S, V \setminus S)$, we let C be the cut induced by $(S, V \setminus S)$ on G . Let $F_i^{(\vec{C})} \stackrel{\text{def}}{=} F_i \cup \vec{C}$ and let $f_i^{(\vec{C})} \stackrel{\text{def}}{=} \mathbf{w}(F_i^{(\vec{C})})$. We also let $F_i^{(C)} \stackrel{\text{def}}{=} F_i \cup C$ and let $f_i^{(C)} \stackrel{\text{def}}{=} \mathbf{w}(F_i^{(C)})$. Lemma 32 shows that for each i , Algorithm 4 achieves β -balanced dicut approximation on subset F_i .

► **Lemma 32.** For any fixed i , with probability at least $1 - \frac{\delta}{2n^2}$, every cut $\vec{C}$ in $\vec{G}$ satisfies

$$\left| f_i^{(\vec{C})} - \tilde{f}_i^{(\vec{C})} \right| \leq \varepsilon \cdot \sqrt{\frac{f_i^{(\vec{C})} f_i^{(C)}}{\beta + 1}} \quad (7)$$

where $\tilde{f}_i^{(\vec{C})} \stackrel{\text{def}}{=} \mathbf{w}_{\vec{H}}(F_i^{(\vec{C})})$.

■ **Algorithm 4** SPARSIFYDICUT($\vec{G} = (V, E, \mathbf{w}), \beta, \varepsilon, \delta, \tilde{\mathbf{k}}$).

-
- 1 **Input:** $\vec{G} = (V, E, \mathbf{w})$ a simple weighted directed graph with $G \stackrel{\text{def}}{=} \text{und}(\vec{G})$, $\beta \in \mathbb{R}_{\geq 0}$, $\varepsilon \in (0, \frac{1}{2})$, $\delta \in (0, 1)$, $\tilde{\mathbf{k}}$ an edge connectivity estimation of G
 - 2 $\rho \stackrel{\text{def}}{=} C_{\text{bal}} \cdot \varepsilon^{-2} (\beta + 1) \log \frac{8n}{\delta}$
 - 3 **foreach** $e \in E$ **do**
 - 4 With probability $p_e = \rho \cdot \frac{\mathbf{w}_e}{\mathbf{k}_e}$, set $\mathbf{w}'_e \leftarrow \frac{1}{p_e} \mathbf{w}_e$; otherwise $\mathbf{w}'_e \leftarrow 0$
 - 5 **return** $\vec{H} = (V, E, \mathbf{w}')$
-

Our proof of Lemma 32 follows by showing using concentration bounds that for each i , the probability of (7) fails is at most $\delta \cdot n^{-\Omega(2^{j+2})}$. Combining this observation with Theorem 31 allows us to obtain the desired probability via a union bound. Then, Theorem 6 follows by applying Cauchy-Schwarz with Lemma 32. See full version for more details.

4.2 Dynamic directed cut sparsification

Since β -balanced dicut approximation satisfies union property (Lemma 28), the reductions presented in Section 3.3 can, again, be applied for dynamic balanced dicut sparsification. This allows us to focus on deriving a decremental algorithm on expanders for this task. Specifically, we observe that Algorithms 2, 3 for dynamically maintaining ε -degree approximations applies here as well.

► **Lemma 33.** *Suppose $\vec{G} = (V, E, \mathbf{w})$ is a weighted directed graph with weight ratio $\frac{\max_{e \in E} \mathbf{w}_e}{\min_{e \in E} \mathbf{w}_e} \leq 2$ undergoing edge deletions only. There is an algorithm that given $\varepsilon, \phi \in (0, 1)$, $\beta \geq 1$ and an oblivious online sequence of edge deletions, preprocesses in time $O(m)$, maintains explicitly a reweighted directed subgraph $\vec{G}'$ of $\vec{G}$ with $\text{nnz}(\mathbf{w}') = O(\varepsilon^2 \beta \phi^{-1} N \log n)$ where N is the number of non-trivial vertices currently. The algorithm has worst-case recourse $O(\varepsilon^{-2} \phi^{-1} \beta \log n)$ and worst-case update time $O(\varepsilon^{-2} \phi^{-1} \beta \log^2 n)$. Furthermore, with high probability, whenever $G = \text{und}(\vec{G})$ becomes a ϕ -expander, $\vec{H}$ is a (β, ε) -dicut sparsifier of $\vec{G}$.*

Theorem 27 follows by applying the dynamic expander decomposition framework in Lemma 19.

5 Directed Spectral Sparsification Against an Adaptive Adversary via Partial Symmetrization

In this section, we consider the problem of fully dynamic degree balance preserving directed spectral sparsification against an adaptive adversary. This is a particularly difficult question, as the state-of-the-art adaptive *undirect* spectral sparsifier requires $O(\log n)$ multiplicative approximation [11], while any current notions of directed spectral sparsification do not permit approximation factor $\varepsilon > 1$. As was observed by Kyng, Meierhans, and Probst Gutenberg [37], a weaker notion of approximation, i.e., approximate pseudoinverse (Definition 7), suffices for solving directed Eulerian Laplacian systems.

The key observation in [37] is that a β -partial symmetrization $\vec{G}^{(\beta)} \stackrel{\text{def}}{=} \beta \cdot G \cup \vec{G}$ of a Eulerian directed graph $\vec{G}$ is a good approximate pseudoinverse of $\vec{G}$.

► **Definition 34** (Generalized partial symmetrization, [37]). *For a directed graph $\vec{G}$ with $G \stackrel{\text{def}}{=} \text{und}(\vec{G})$ and $\beta \geq 0$, we let $\vec{G}^{(\beta)} \stackrel{\text{def}}{=} \beta \cdot G \cup \vec{G}$ be the β -partial symmetrization of $\vec{G}$.*

We remark that $\vec{L}_{\vec{G}^{(\beta)}} = \beta \mathbf{L}_G + \vec{L}_{\vec{G}}$ and $\mathbf{L}_{G^{(\beta)}} = (2\beta + 1) \mathbf{L}_G$ where $G^{(\beta)} = \text{und}(\vec{G}^{(\beta)})$. Subsequently, one can sparsify the directed portion of $\vec{G}^{(\beta)}$ much more aggressively under the notion of degree balance preserving spectral approximation, since

$$\left\| \mathbf{L}_{G^{(\beta)}}^{\frac{1}{2}} (\vec{L}_{\vec{G}} - \vec{L}_{\vec{H}}) \mathbf{L}_{G^{(\beta)}}^{\frac{1}{2}} \right\|_{\text{op}} = \frac{1}{2\beta + 1} \left\| \mathbf{L}_G^{\frac{1}{2}} (\vec{L}_{\vec{G}} - \vec{L}_{\vec{H}}) \mathbf{L}_G^{\frac{1}{2}} \right\|_{\text{op}}.$$

Moreover, the graph $\beta G \cup \vec{H}$ is also a good approximate pseudoinverse of $\vec{G}^{(\beta)}$. One can further reduce the size of the graph by performing an undirected spectral sparsification of the undirected portion βG . These layers of sparsification allows them to build sparse preconditioner chain which can be used in solving directed Eulerian Laplacian systems by applying an iterative solver, such as Preconditioned Richardson (see Lemma 35). We refer readers to [37] and [20] for further expositions of this idea.

► **Lemma 35** (Preconditioned Iterations, Lemma 4.4 in [20]). *If $\mathbf{Z}$ is a λ -approximate pseudoinverse of $\mathbf{M}$ with respect to $\mathbf{U}$ for $\lambda \in (0, 1)$, $\mathbf{b} \perp \ker(\mathbf{M}^\top)$ and $N \geq 0$, then $\mathbf{x}_N = \text{PRECONRICHARDSON}(\mathbf{M}, \mathbf{Z}, \mathbf{b}, N)$ computes $\mathbf{x}_N = \mathbf{Z}_N \mathbf{b}$ for some matrix $\mathbf{Z}_N$ only depending on $\mathbf{Z}, \mathbf{M}$, and N , such that $\mathbf{Z}_N$ is a λ^N -approximate pseudoinverse of $\mathbf{M}$ with respect to $\mathbf{U}$.*

Following [37], we consider for $\vec{G}_0 = \vec{G}$, three graphs $\vec{G}_1, \vec{G}_2, \vec{G}_3$, such that each $\vec{G}_{i+1}$ spectrally approximates $\vec{G}_i$. Specifically,

1. $\vec{G}_1 = \vec{G}_0^{(\beta)}$ is the β -partial symmetrization of $\vec{G}_0$,
2. $\vec{G}_2 = \beta \cdot G \cup \vec{H}_2$ is a sparsification of the directed portion $\vec{G}$ of $\vec{G}_1$,
3. $\vec{G}_3 = \beta \cdot H_3 \cup \vec{H}_2$, the final sparsifier, further sparsifies the undirected portion of $\beta \cdot G$ of $\vec{G}_2$.

We now recall the definition of a *sparsification quadruple*.

► **Definition 36** (Sparsification quadruple, [37]). *We call strongly connected Eulerian graphs $\vec{G}_0, \vec{G}_1, \vec{G}_2, \vec{G}_3$ a (γ, β, α) -quadruple if*

1. $\vec{L}_{\vec{G}_i}^\dagger$ is a $1 - \frac{1}{\gamma}$ -approximate pseudoinverse of $\vec{L}_{\vec{G}_{i-1}}$ with respect to $\mathbf{L}_{G_i}$ for $i = 1, 2, 3$,
2. $\frac{1}{\gamma} \mathbf{L}_{G_{i-1}} \preceq \mathbf{L}_{G_i} \preceq \gamma \mathbf{L}_{G_{i-1}}$ for $i = 1, 2, 3$,
3. $\mathbf{d}_{\vec{G}_0}^{(i)} = \mathbf{d}_{\vec{G}_0}^{(o)} = (1 + \beta) \mathbf{d}_{\vec{G}_i}^{(i)} = (1 + \beta) \mathbf{d}_{\vec{G}_i}^{(o)}$ for $i = 1, 2$, and $\mathbf{d}_{\vec{G}_0}^{(i)} = (1 + \frac{\beta}{\alpha}) \mathbf{d}_{\vec{G}_3}^{(i)} = (1 + \frac{\beta}{\alpha}) \mathbf{d}_{\vec{G}_3}^{(o)}$,
4. $\vec{G}_3$ is nearly-linear in n and $m(\vec{G}_i) = O(m)$ for $i = 1, 2$.

Our adaptive degree (balance) preserving directed spectral sparsification algorithm maintains explicitly a sparsification quadruple. Similar to Theorem 3, our sparsifiers in Theorem 8 contain a few extra vertices and the approximation guarantees are given for the Schur complements with those vertices removed. Having such extra vertices does not affect preconditioning, as we observed in Lemma 13. When the input graph $\vec{G}$ becomes Eulerian, for a constant approximate pseudoinverse of $\vec{G}_3$, we can solve the system of linear equation using at most $\text{polylog}(n, \varepsilon^{-1})$ applications of the pseudoinverse and each matrix $\vec{L}_{\vec{G}_i}$ for $i = 1, 2, 3$ using Lemma 35.

5.1 Sparsification quadruples

In this section, we consider the approximation factors between each $\vec{G}_i$ and $\vec{G}_{i+1}$ under the notion of approximate pseudoinverse.

► **Lemma 37** (Lemma 4.6, [37]). *For Eulerian $\vec{G}_1 = \vec{G}_0^{(\beta)}$, the matrix $\vec{L}_{\vec{G}_1}^\dagger$ is an $(1 - \frac{1}{1+2\beta})$ -approximate pseudoinverse of $\vec{L}_{\vec{G}_0}$ with respect to $\mathbf{L}_{G_1}$ for $G_1 \stackrel{\text{def}}{=} \text{und}(\vec{G}_1)$.*

Lemma 38 states that if $\vec{G}_2$ is a good degree balance preserving spectral approximate to $\vec{G}_1$, then it is also a good approximate pseudoinverse.

► **Lemma 38.** *For $\beta > 0$, $\varepsilon \in (0, 1)$ and Eulerian $\vec{G}$, if $\vec{G}_2$ is a ε -degree balance preserving directed spectral approximation of $\vec{G}_1$ and is Eulerian, then $\vec{L}_{\vec{G}_2}^\dagger$ is a $\frac{\varepsilon \cdot (2\beta+1)}{\beta}$ -approximate pseudoinverse of $\vec{L}_{\vec{G}_1}$ with respect to $\mathbf{L}_{\vec{G}_2}$ and $(1 - \frac{1}{2\beta+1}) \cdot \mathbf{L}_{G_1} \preceq \mathbf{L}_{G_2} \preceq (1 + 2\varepsilon) \cdot \mathbf{L}_{G_1}$.*

Similarly, Lemma 40 states that using a good multiplicative undirected spectral approximation to G , one can generate $\vec{G}_3$ that is also a good approximate pseudoinverse of $\vec{G}_2$. Here, we recall the definition of multiplicative undirected spectral approximation.

► **Definition 39** (Undirected spectral approximation, multiplicative). *$H = (V, E_H, \mathbf{w}_H)$ is a γ -spectral approximation of $G = (V, E, \mathbf{w})$ if it satisfies for some scalar η that*

$$\mathbf{L}_G \preceq \eta \mathbf{L}_H \preceq \gamma \cdot \mathbf{L}_G.$$

► **Lemma 40.** *For $\lambda > 1$ and Eulerian $\vec{G}$, if the graph H satisfies $\mathbf{L}_G \preceq \mathbf{L}_H \preceq \lambda \mathbf{L}_G$, then $\vec{G}_3 = \frac{1}{2}\beta(\lambda+1)\mathbf{L}_H + \vec{H}_2$ satisfies that $\vec{L}_{\vec{G}_3}^\dagger$ is a $(1 - \frac{2}{\lambda+1})$ -approximate pseudoinverse of $\vec{L}_{\vec{G}_2}$ with respect to $\mathbf{L}_{G_3}$ and $\mathbf{L}_{G_2} \preceq \mathbf{L}_{G_3} \preceq \frac{1}{2}\lambda(\lambda+1) \cdot \mathbf{L}_{G_2}$.*

5.2 Dynamic reductions to pruned almost uniform degree expanders

We will consider a variant of expanders where the minimum degree is guaranteed to be close to the average degree. We say that a graph $G = (V, E, \mathbf{w})$ is (ϕ, γ) -uniform degree expander if G is a ϕ -expander and that $\frac{\max_{v \in V} \mathbf{d}_v}{\min_{v \in V} \mathbf{d}_v} \leq \gamma^{-1}$, where we denote $\mathbf{d}_v = \sum_{e \in E | e \ni v} \mathbf{w}_e$ as the weighted degree of v . Recall that we assume the subgraphs in a decomposition contain the *entire set of edges*.

An uniform degree expander decomposition of a graph is defined similarly to Definition 16 but with a few additional requirements on the degrees. For each subgraph $\vec{G}_i$, it is associated with a graph $\vec{G}'_i$ supported on the vertices of $\vec{G}_i$ and extra sets of vertices $\{Y_{v,i}\}$ for $v \in V(\vec{G}_i)$. The graph $\vec{G}'_i$ is required to be a (ϕ, γ) -uniform degree expander satisfying that contracting each $Y_{v,i}$ in $\vec{G}'_i$ gives $\vec{G}_i$.

Informally speaking, Theorem 5.8 in [11] provide a similar algorithm to Proposition 18 where an uniform degree expander decomposition of a weighted dynamic graph can be maintained efficiently where each expander undergoes only decremental updates. In the full version, we provide a slight modification for directed graphs and with edge weight bucketing.

We now recall the reduction to decremental algorithm on uniform degree expanders by [11] using this dynamic uniform degree expander decomposition algorithm from above. The version we present here is an informal version of Theorem 5.2 in [11] with a slight generalization: we remove the perturbation condition and allows for directed graphs.

► **Lemma 41** (Informal blackbox adaptive reduction with uniform degree promise). *Assume $\mathcal{H}$ is a graph sparsification problem that satisfies the union and contraction property, and there is a decremental algorithm $\mathcal{A}$ for $\mathcal{H}$ on (directed) graphs with edge weights ratio at most 2 and satisfying that their corresponding undirected graph is a uniform degree expander.*

Then, there exists a fully dynamic algorithm $\mathcal{B}$ for $\mathcal{H}$ on weighted (directed) graphs such that given a dynamic weighted simple directed graph $\vec{G}$ maintains an ε -approximate sparsifier of essentially the same size with essentially the same amortized update time as $\mathcal{A}$. Moreover, $\mathcal{B}$ works against an adaptive adversary if $\mathcal{A}$ supports adaptive edge updates.

5.3 Adaptive degree preserving sparsifications

In this section, we provide our guarantees for dynamic sparsification from $\vec{G}_i$ to $\vec{G}_{i+1}$.

Theorem 42 gives dynamic sparsification from $\vec{G}_2$ to $\vec{G}_3$, which, by Lemma 40, reduces to the sparsification of the undirected graph $G = \text{und}(\vec{G})$ against an adaptive adversary.

► **Theorem 42.** *There is a fully dynamic algorithm that given as input a weighted undirected graph $G = (V, E, \mathbf{w})$ with bounded weight ratio W at all time undergoing adaptive edge insertions and deletions, maintain explicitly a graph $\vec{H}$ on vertices $V \cup X$ with X disjoint from V . The Schur complement $S = \text{Sc}(\vec{H}, V)$ is an undirected graph and satisfies with high probability that S is $O(\log^{12} n)$ -degree preserving spectral approximation of G . The graph $\vec{H}$ has size $\tilde{O}(n \log W)$ and extra number of vertices $|X| \leq O(n \log^3 n \log W)$. The algorithm has preprocessing time $\tilde{O}(m)$ and amortized update time $\tilde{O}(1)$.*

For sparsification from $\vec{G}_1$ to $\vec{G}_2$, by Lemma 38, it suffices to produce a ε -degree preserving spectral sparsification $\vec{H}$ of $\vec{G}$ with respect to $\mathbf{L}_{G^{(\beta)}}$. The exact statement of our guarantees is presented in Lemma 43.

► **Lemma 43.** *Let $\vec{G} = (V, E, \mathbf{w})$ be a dynamic weighted directed graph undergoing adaptive edge updates. There is a deterministic dynamic algorithm that for $\varepsilon \in (0, 1)$, if $\beta \geq \Omega(\varepsilon^{-1} \log^4 n)$, it maintains explicitly a weighted directed graph $\vec{H} = (V \cup X, E_{\vec{H}}, \mathbf{w}_{\vec{H}})$*

satisfying that for Schur complement $\vec{S} = \text{Sc}(\vec{H}, V)$ $\vec{G}' = \beta \cdot G \cup \vec{S}$ is a ε -degree preserving directed spectral sparsifier of $\vec{G}^{(\beta)}$, where $G = \text{und}(\vec{G})$. The graph $\vec{H}$ has size $\tilde{O}(n \log W)$ and extra number of vertices $|X| \leq O(n \log^3 n \log W)$. The algorithm has preprocessing time $\tilde{O}(m)$ and amortized update time $\tilde{O}(1)$.

Our algorithm in Theorem 8 maintains (a) a copy of the algorithm from Lemma 43 for sparsification from $\vec{G}_1$ to $\vec{G}_2$, and (b) a copy of the algorithm from Theorem 42 for sparsification from $\vec{G}_2$ to $\vec{G}_3$ with the sparsifier scaled appropriately. Note that (a) maintains a graph $\vec{H}_2$ on $V \cup X$ and (b) maintain a graph $\vec{H}_3$ on $V \cup X'$ for some subset X' disjoint from X . By the disjointness, we have the following equivalence in the Schur complement,

$$\text{Sc}(\vec{G}_3, V) = \text{Sc}(\vec{H}_3 \cup \vec{H}_2, V) = \text{Sc}(\vec{H}_3, V) \cup \text{Sc}(\vec{H}_2, V).$$

Then, the union property (Lemma 11) then ensures the Schur complement $\text{Sc}(\vec{H}_3 \cup \vec{H}_2, H)$ is a degree preserving spectral approximation to $\vec{G}_2$. Theorem 8 follows by picking appropriate parameters in Theorem 42 and Lemma 43.

To prove Theorem 42, we use the observation that a degree preserving undirected cut sparsifier of an expander is a good spectral to the expander as well (Lemma 45). This observation allows to utilize an adaptive algorithm for cut sparsification from [11] on decremental uniform degree expanders.

► **Definition 44** (Undirected cut approximation, multiplicative). $H = (V, E_H, \mathbf{w}_H)$ is a γ -cut approximation of $G = (V, E, \mathbf{w})$ if for all non-trivial $U \subseteq V$, it satisfies that

$$\mathbf{w}(U, V \setminus U) \leq \mathbf{w}_H(U, V \setminus U) \leq \gamma \cdot \mathbf{w}(U, V \setminus U).$$

► **Lemma 45** ([17, 11]). Let G, H be weighted undirected graphs with the same set of vertices and satisfying $\mathbf{d}_G = \mathbf{d}_H$. If both G, H are ϕ -expanders for $\phi \in (0, 1]$, then

$$\frac{\phi^2}{4} \mathbf{L}_G \preceq \mathbf{L}_H \preceq \frac{4}{\phi^2} \mathbf{L}_G.$$

We observe that a degree preserving spectral sparsifier of an almost uniform degree expander is easy to compute using cut sparsifiers and our patching scheme in Lemma 46. Note that our returned graph $\vec{H}$ on vertices $V \cup X$ is instead *directed*. This is not a problem: (1) our overall sparsifier $\vec{G}_3$ is ultimately a directed graph, and (2) the Schur complement $S = \text{Sc}(\vec{H}, V)$ back onto the original set of vertices is an undirected graph. We further notice that Lemma 46 require specifically for the graph G to be a bipartite graph. This is an important assumption to ensure our star patching does not generate self-loops after “unlifting” the graph. See Section 3.6 for the exact reduction.

► **Lemma 46.** Let weighted bipartite undirected graph $G = (C \cup R, E, \mathbf{w})$ be a ϕ -expander. If $\tilde{G}$ is a γ -cut approximation of G , then there is a deterministic algorithm that computes in time $O(m_G + m_{\tilde{G}} + n \log n)$ a directed graph $\vec{H}$ on vertices $V \cup \{x, y\}$ for $V = C \cup R$ such that the Schur complement $S = \text{Sc}(\vec{H}, V)$ is an undirected graph that satisfies $\mathbf{d}_S = \mathbf{d}_G$, and $\frac{4\gamma^2}{\phi^2} \cdot S$ is a $\frac{16\gamma^4}{\phi^4}$ -spectral approximation of G . The graph H has size $m_H = O(m_{\tilde{G}} + n)$,

We can now apply a decremental cut sparsification algorithm from [11] for spectral sparsification. Remark that the following Lemma 47 has a slight modification to allow for edge weights to be within a factor of 2, instead of uniform. Theorem 42 is then a direct consequence when combined with Lemma 46.

► **Lemma 47** (Decremental adaptive cut sparsifier, Theorem 9.5 [11]). *Let $G = (V, E, \mathbf{w})$ be a decremental weighted graph undergoing adaptive edge deletions that satisfies $L \leq \mathbf{w}_e \leq 2L$ for all e and $\Delta_{\max} \geq \mathbf{d}_v \geq \Delta_{\min} \geq \frac{80 \log n}{\phi}$ for some fixed ϕ, L , where $\Delta_{\max}$ is the maximum weighted degree in the initial graph G and $\Delta_{\min}$ is a degree lowerbound. Then, there is an algorithm that maintains a reweighted subgraph H of G such that $m_H = \tilde{O}\left(n \left(\frac{\Delta_{\max}}{\phi \Delta_{\min}}\right)^2\right)$ and at any stage where H has $\min_{v \in V} \mathbf{d}_v \geq \Delta_{\min}$ and $\phi(H) \geq \phi$ on non-trivial vertices, H is a $O(\log n)$ -approximate cut sparsifier of H with high probability. The algorithm has preprocessing time $O(m)$ and worst-case update time $\tilde{O}\left(\left(\frac{\Delta_{\max}}{\phi \Delta_{\min}}\right)^3\right)$.*

Consider sparsification from $\vec{G}_1$ to $\vec{G}_2$. Similar to the undirected sparsification, a directed graph $\vec{H}$ can be computed using a star patching for expanders. We again consider a bipartite graph with “disjoint” vertices to avoid self-loops when eliminating the extra vertices. Lemma 43 can then be proven straightforwardly using Lemma 48.

► **Lemma 48.** *Let $\vec{G} = (C \cup R, E, \mathbf{w})$ be a weighted bipartite directed graph with $E \subseteq C \times R$ so that $G = \text{und}(\vec{G})$ is a ϕ -expander. There is a deterministic algorithm that given $\varepsilon \in (0, 1)$, and β satisfying $\beta \geq 2\varepsilon^{-1}\phi^{-2}$, returns in time $O(m + n \log n)$ a graph $\vec{H}$ on vertices $C \cup R \cup \{x\}$ of size $O(n)$ such that for $\vec{G}' = \beta \cdot G \cup \text{Sc}(\vec{H}, C \cup R)$ is a ε -degree preserving directed spectral sparsifier of $\vec{G}^{(\beta)}$.*

References

- 1 Ittai Abraham, David Durfee, Ioannis Koutis, Sebastian Krinninger, and Richard Peng. On fully dynamic graph sparsifiers. In *2016 IEEE 57th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 335–344, 2016. doi:10.1109/FOCS.2016.44.
- 2 Daniel Agassy, Dani Dorfman, and Haim Kaplan. Expander decomposition with fewer inter-cluster edges using a spectral cut player. In *50th International Colloquium on Automata, Languages, and Programming, ICALP 2023, July 10–14, 2023, Paderborn, Germany*, volume 261 of *LIPIcs*, pages 9:1–9:20. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023. doi:10.4230/LIPIcs.ICALP.2023.9.
- 3 AmirMahdi Ahmadinejad, Arun Jambulapati, Amin Saberi, and Aaron Sidford. *Perron-Frobenius Theory in Nearly Linear Time: Positive Eigenvectors, M-matrices, Graph Kernels, and Other Applications*, pages 1387–1404. SIAM, 2019. doi:10.1137/1.9781611975482.85.
- 4 AmirMahdi Ahmadinejad, Jonathan A. Kelner, Jack Murtagh, John Peebles, Aaron Sidford, and Salil P. Vadhan. High-precision estimation of random walks in small space. In *61st IEEE Annual Symposium on Foundations of Computer Science, FOCS*, pages 1295–1306. IEEE, 2020. doi:10.1109/FOCS46700.2020.00123.
- 5 AmirMahdi Ahmadinejad, John Peebles, Edward Pyne, Aaron Sidford, and Salil P. Vadhan. Singular value approximation and sparsifying random walks on directed graphs. In *64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023*, pages 846–854. IEEE, 2023. doi:10.1109/FOCS57990.2023.00054.
- 6 Giorgio Ausiello, Paolo Franciosa, and Giuseppe Italiano. Small stretch spanners on dynamic graphs. *Journal of Graph Algorithms and Applications*, 10(2):365–385, 2006. doi:10.7155/JGAA.00133.
- 7 Nikhil Bansal, Haotian Jiang, and Raghu Meka. Resolving matrix spencer conjecture up to poly-logarithmic rank. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023*, pages 1814–1819. ACM, 2023. doi:10.1145/3564246.3585103.
- 8 Surender Baswana, Sumeet Khurana, and Soumojit Sarkar. Fully dynamic randomized algorithms for graph spanners. *ACM Trans. Algorithms*, 8(4), October 2012. doi:10.1145/2344422.2344425.

- 9 András A. Benczúr and David R. Karger. Approximating s-t minimum cuts in $\tilde{O}(n^2)$ time. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '96, pages 47–55. Association for Computing Machinery, 1996. doi:10.1145/237814.237827.
- 10 Aaron Bernstein, Sebastian Forster, and Monika Henzinger. A deamortization approach for dynamic spanner and dynamic maximal matching. *ACM Trans. Algorithms*, 17(4), October 2021. doi:10.1145/3469833.
- 11 Aaron Bernstein, Jan van den Brand, Maximilian Probst Gutenberg, Danupon Nanongkai, Thatchaphol Saranurak, Aaron Sidford, and He Sun. Fully-Dynamic Graph Sparsifiers Against an Adaptive Adversary. In Mikołaj Bojańczyk, Emanuela Merelli, and David P. Woodruff, editors, *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*, volume 229 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 20:1–20:20, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ICALP.2022.20.
- 12 Karl Bringmann and Konstantinos Panagiotou. Efficient sampling methods for discrete distributions. In Artur Czumaj, Kurt Mehlhorn, Andrew Pitts, and Roger Wattenhofer, editors, *Automata, Languages, and Programming*, pages 133–144, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg. doi:10.1007/978-3-642-31594-7_12.
- 13 Ruoxu Cen, Yu Cheng, Debmalya Panigrahi, and Kevin Sun. Sparsification of Directed Graphs via Cut Balance. In Nikhil Bansal, Emanuela Merelli, and James Worrell, editors, *48th International Colloquium on Automata, Languages, and Programming (ICALP 2021)*, volume 198 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 45:1–45:21, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ICALP.2021.45.
- 14 Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva. Maximum flow and minimum-cost flow in almost-linear time. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 612–623, 2022. doi:10.1109/FOCS54457.2022.00064.
- 15 Yu Cheng, Max Li, Honghao Lin, Zi-Yi Tai, David P. Woodruff, and Jason Zhang. Tight lower bounds for directed cut sparsification and distributed min-cut. *Proc. ACM Manag. Data*, 2(2), May 2024. doi:10.1145/3651148.
- 16 Timothy Chu, Yu Gao, Richard Peng, Sushant Sachdeva, Saurabh Sawlani, and Junxing Wang. Graph sparsification, spectral sketches, and faster resistance computation, via short cycle decompositions. In *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 361–372, 2018. doi:10.1109/FOCS.2018.00042.
- 17 Julia Chuzhoy, Yu Gao, Jason Li, Danupon Nanongkai, Richard Peng, and Thatchaphol Saranurak. A deterministic algorithm for balanced cut with applications to dynamic connectivity, flows, and beyond. In *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1158–1167, 2020. doi:10.1109/FOCS46700.2020.00111.
- 18 Julia Chuzhoy and Merav Parter. *Fully Dynamic Algorithms for Graph Spanners via Low-Diameter Router Decomposition*, pages 785–823. SIAM, 2025. doi:10.1137/1.9781611978322.23.
- 19 Michael B Cohen, Jonathan Kelner, Rasmus Kyng, John Peebles, Richard Peng, Anup B Rao, and Aaron Sidford. Solving directed laplacian systems in nearly-linear time through sparse lu factorizations. In *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 898–909. IEEE, 2018. doi:10.1109/FOCS.2018.00089.
- 20 Michael B Cohen, Jonathan Kelner, John Peebles, Richard Peng, Anup B Rao, Aaron Sidford, and Adrian Vladu. Almost-linear-time algorithms for markov chains and new spectral primitives for directed graphs. In *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*, pages 410–419, 2017. doi:10.1145/3055399.3055463.
- 21 Michael B Cohen, Jonathan Kelner, John Peebles, Richard Peng, Aaron Sidford, and Adrian Vladu. Faster algorithms for computing the stationary distribution, simulating random walks, and more. In *2016 IEEE 57th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 583–592. IEEE, 2016. doi:10.1109/FOCS.2016.69.

- 22 Luc Devroye. Chapter 4 nonuniform random variate generation. In Shane G. Henderson and Barry L. Nelson, editors, *Simulation*, volume 13 of *Handbooks in Operations Research and Management Science*, pages 83–121. Elsevier, 2006. doi:10.1016/S0927-0507(06)13004-2.
- 23 Michael Elkin. Streaming and fully dynamic centralized algorithms for constructing and maintaining sparse spanners. *ACM Trans. Algorithms*, 7(2), March 2011. doi:10.1145/1921659.1921666.
- 24 Alina Ene, Gary Miller, Jakub Pachocki, and Aaron Sidford. Routing under balance. In *Proceedings of the Forty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '16, pages 598–611, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/2897518.2897654.
- 25 Sebastian Forster and Gramoz Goranci. Dynamic low-stretch trees via dynamic low-diameter decompositions. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2019, pages 377–388, New York, NY, USA, 2019. Association for Computing Machinery. doi:10.1145/3313276.3316381.
- 26 Sebastian Forster, Gramoz Goranci, Yang P Liu, Richard Peng, Xiaorui Sun, and Mingquan Ye. Minor sparsifiers and the distributed laplacian paradigm. In *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 989–999. IEEE, 2021. doi:10.1109/FOCS52979.2021.00099.
- 27 Wai Shing Fung, Ramesh Hariharan, Nicholas J.A. Harvey, and Debmalaya Panigrahi. A general framework for graph sparsification. In *Proceedings of the Forty-Third Annual ACM Symposium on Theory of Computing*, STOC '11, pages 71–80, New York, NY, USA, 2011. Association for Computing Machinery. doi:10.1145/1993636.1993647.
- 28 David Gibb, Bruce Kapron, Valerie King, and Nolan Thorn. Dynamic graph connectivity with improved worst case update time and sublinear space, 2015. arXiv:1509.06464.
- 29 Gramoz Goranci, Monika Henzinger, Harald Räcke, and A. R. Sricharan. Incremental Approximate Maximum Flow via Residual Graph Sparsification. In Keren Censor-Hillel, Fabrizio Grandoni, Joël Ouaknine, and Gabriele Puppis, editors, *52nd International Colloquium on Automata, Languages, and Programming (ICALP 2025)*, volume 334 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 91:1–91:20, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ICALP.2025.91.
- 30 Jacob Holm, Eva Rotenberg, and Christian Wulff-Nilsen. Faster fully-dynamic minimum spanning forest. In Nikhil Bansal and Irene Finocchi, editors, *Algorithms - ESA 2015*, pages 742–753, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg. doi:10.1007/978-3-662-48350-3_62.
- 31 Arun Jambulapati, Victor Reis, and Kevin Tian. *Linear-Sized Sparsifiers via Near-Linear Time Discrepancy Theory*, pages 5169–5208. SIAM, 2024. doi:10.1137/1.9781611977912.186.
- 32 Arun Jambulapati, Sushant Sachdeva, Aaron Sidford, Kevin Tian, and Yibin Zhao. *Eulerian Graph Sparsification by Effective Resistance Decomposition*, pages 1607–1650. SIAM, 2025. doi:10.1137/1.9781611978322.50.
- 33 Bruce M. Kapron, Valerie King, and Ben Mountjoy. *Dynamic graph connectivity in polylogarithmic worst case time*, pages 1131–1142. SIAM, 2013. doi:10.1137/1.9781611973105.81.
- 34 David R. Karger and Matthew S. Levine. Fast augmenting paths by random sampling from residual graphs. *SIAM Journal on Computing*, 44(2):320–339, 2015. doi:10.1137/070705994.
- 35 Donald E. Knuth. *The art of computer programming, volume 2 (3rd ed.): seminumerical algorithms*. Addison-Wesley Longman Publishing Co., Inc., USA, 1997.
- 36 Ioannis Koutis and Shen Chen Xu. Simple parallel and distributed algorithms for spectral graph sparsification. *ACM Trans. Parallel Comput.*, 3(2), 2016. doi:10.1145/2948062.
- 37 Rasmus Kyng, Simon Meierhans, and Maximilian Probst Gutenberg. Derandomizing directed random walks in almost-linear time. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 407–418, 2022. doi:10.1109/FOCS54457.2022.00046.
- 38 Lap Chi Lau, Robert Wang, and Hong Zhou. *Spectral Sparsification by Deterministic Discrepancy Walk*, pages 315–340. SIAM, 2025. doi:10.1137/1.9781611978315.24.

- 39 Huan Li and Aaron Schild. Spectral subspace sparsification. In *59th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2018*, pages 385–396. IEEE Computer Society, 2018. doi:10.1109/FOCS.2018.00044.
- 40 Yang P Liu, Sushant Sachdeva, and Zejun Yu. Short cycles via low-diameter decompositions. In *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 2602–2615. SIAM, 2019. doi:10.1137/1.9781611975482.161.
- 41 Merav Parter and Eylon Yogev. Optimal Short Cycle Decomposition in Almost Linear Time. In *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*, volume 132 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 89:1–89:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019. doi:10.4230/LIPIcs.ICALP.2019.89.
- 42 Richard Peng and Zhuoqing Song. Sparsified block elimination for directed laplacians. In *STOC '22: 54th Annual ACM SIGACT Symposium on Theory of Computing*, pages 557–567. ACM, 2022. doi:10.1145/3519935.3520053.
- 43 Lucas Pesenti and Adrian Vladu. *Discrepancy Minimization via Regularization*, pages 1734–1758. SIAM, 2023. doi:10.1137/1.9781611977554.ch66.
- 44 Sushant Sachdeva, Anvith Thudi, and Yibin Zhao. Better Sparsifiers for Directed Eulerian Graphs. In Karl Bringmann, Martin Grohe, Gabriele Puppis, and Ola Svensson, editors, *51st International Colloquium on Automata, Languages, and Programming (ICALP 2024)*, volume 297 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 119:1–119:20, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ICALP.2024.119.
- 45 Thatchaphol Saranurak and Di Wang. Expander decomposition and pruning: Faster, stronger, and simpler. In Timothy M. Chan, editor, *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2019*, pages 2616–2635. SIAM, 2019. doi:10.1137/1.9781611975482.162.
- 46 Daniel Spielman and Nikhil Srivastava. Graph sparsification by effective resistances. *SIAM Journal on Computing*, 40(6):1913–1926, 2011. doi:10.1137/080734029.
- 47 Jan van den Brand, Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, Sushant Sachdeva, and Aaron Sidford. A deterministic almost-linear time algorithm for minimum-cost flow. In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 503–514, 2023. doi:10.1109/FOCS57990.2023.00037.
- 48 Yibin Zhao. Fully dynamic spectral and cut sparsifiers for directed graphs, 2025. doi:10.48550/arXiv.2507.19632.